Computer Programming for Civil Engineering

Dr. Bilal Tayfur

February 14, 2026

This document is prepared for students of the Civil Engineering Department. The Python programming language is used in this course. However, the course is not about Python programming. It is about computer programming in general. In addition, some other programming languages are also used to demonstrate the some specific concepts. The main objective of this course is to introduce students to the basic concepts. Python programming language is selected because of its simplicity and ease of use. If you are familiar with any other programming language, you can use it to solve all exercises. The AI usage is allowed for students. However, in learning process, its not recommended.

With recent developments in computer science, everything changes rapidly. So, the things that will be mentioned here only valid for a short period of time. However, if you are focused on learing programming for engineering purposes, this course and Python language will be useful for you. Python makes some concepts (such as variables, arrays, data types etc.) easier to understand. But if you are interested in learning programming for "pure programming" purposes, maybe you should start with other programming languages. At some point, you can switch your focus to Python, anyway, the previous knowledge that you gain from more complex programming languages will be very useful for you.

Additional Resources

- https://www.python.org/
- https://books.goalkicker.com/PythonBook/PythonNotesForProfessionals.pdf
- https://projecteuler.net/
- Kutlu Darılmaz, Yapı Mühendisliği Örnekleriyle C++ Kullanımına Giriş, Birsen Yayınevi

Contents

Basic rules

Decimal points are always "." Most of the programming languages use "." as the decimal point. This is strict rule and there is a reason for this. The comma is usually used as parameter separator in many programming languages. If we use the comma as decimal point, it will create a lot of confusion.

1 Basic concepts of computer programming

1.1 What is computer programming?

Computer programming (or coding) is the process of designing and building an executable computer program to accomplish a specific computing result. It involves writing instructions in a language that computers—or, more accurately, interpreters or compilers—can understand. Think of it as writing a recipe: you list the ingredients (data) and the step-by-step instructions (algorithms) to cook a meal (the solution).

1.2 Why computer programming?

Programming empowers us to:

1. **Automate Tasks**: Perform repetitive calculations (e.g., calculating the area of 1000 different foundations) instantly.
2. **Solve Complex Problems**: Handle calculations that are too time-consuming or error-prone for manual work, such as Finite Element Analysis (FEA).
3. **Process Data**: Analyze vast amounts of data collected from sensors or experiments.

1.3 How to learn computer programming?

Learning to program is like learning a new spoken language or a musical instrument. It requires:

- **Practice**: Writing code is the only way to learn code.
- **Problem Solving**: Breaking down a big problem into smaller, manageable steps.
- **Patience**: Debugging (fixing errors) is a major part of the process.

Note

Reading code is as important as writing it. Try to understand what every line of an example does before running it.

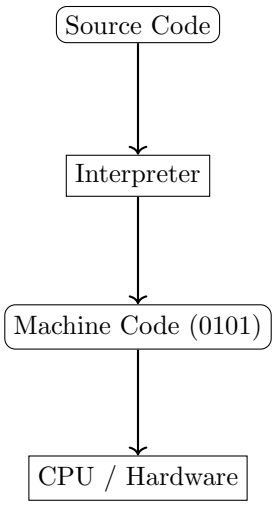
1.4 Why we use programming languages in engineering?

In Civil Engineering, we use programming to:

- Develop design tools (e.g., converting a spreadsheet calculation into a robust application).
- Customize commercial software (e.g., using API in SAP2000, Ansys or Plaxis).
- Perform research (e.g., developing new material models or seismic analysis algorithms).

1.5 How programming languages work?

Computers only understand "Machine Code" (binary: 0s and 1s). Programming languages allow us to write in a human-readable format (Source Code). A translation layer (Compiler or Interpreter) then converts our code into machine instructions.



The performance differences between programming languages are directly related the interpretion process. Usually, the lower the level of the language, the faster it is. So the Python language is not the best choice in terms of performance. However, with the development of the hardwares, the performance differences between programming languages are becoming less and less significant, especially for the basic or intermediate level programming tasks.

2 Introduction to Python

2.1 What is Python?

Python is a high-level, interpreted programming language known for its easy readability and vast ecosystem of libraries. It was created by Guido van Rossum and released in 1991 (It is much older than you might think). It emphasizes code readability with its notable use of significant indentation. The indentation in Python is not just for the sake of looks, it is a part of the syntax of the language. Actually, this concept is very easy to understand, however, for a beginner, it can be a bit confusing. When you start to write and read as much as you can, you will get used to it. Every programming language has its own syntax, and Python is no exception. For Python or any other programming language, this specific rules and some exceptions are needed to be learned. This learning curve is different for every programming language. Python is very suitable for beginners, but to be an expert, you need to practice a lot.

When we talk about Python, the concept of "scripting language" should be mentioned. The other concept is "programming language". In detail, the difference between two of them is not quite easy to explain. However, in short, scripting languages are usually interpreted, while programming languages are usually compiled. The practical difference between two of them is not quite significant for most of the cases for in-house developers.

Note

The level concept in programming languages usually misundertood by beginners. It does not mean the difficulty of the language, but the complexity of the code. The level, here, means the similarity to human language. If a language is more similar to human language, we call this language "high-level". If a language is more similar to machine language, we call this language "low-level". For example, "Python" is a high-level language, while "Assembly" is a low-level language.

2.2 Why Python?

For engineering purposes, Python is the top choice because:

1. **Readability**: Python code looks like pseudo-code or simple English.
2. **Libraries**: It has powerful libraries for engineering:
   - `NumPy`: For numerical computations (matrices, algebra).
   - `Pandas`: For data manipulation.
   - `Matplotlib`: For plotting graphs.
   - `OpenSeesPy`: For earthquake engineering simulations.
3. **Community**: A huge community means you can find answers to almost any problem online. That community shouldn't be underestimated. Because of this popularity, these huge libraries are always getting wider and wider. Now you can find almost any library for a specific purpose.

2.3 How to install Python?

The most common way to install Python is now a distribution called **Anaconda**. It is a distribution that includes Python and most of the scientific libraries we need. But for beginners, it can be a bit overwhelming. So because of this, you can start with only Python and a simple IDE like VSCode, Cursor, Antigravity. This approach will be useful also because of the easy usage of AI tools.

For Windows users, you can install Python using the following command:

```
winget install python
```

For Linux users, you can install Python using the following command:

```
sudo apt install python3
```

For Mac users, you can install Python using the following command (Homebrew is required):

```
brew install python
```

2.4 How to run Python?

You can run Python code in several ways: 1. **Interactive Mode**: Typing commands one by one in a terminal. 2. **Script Mode**: Writing code in a text file (with `.py` extension) and running the file. 3. **Notebooks**: Using Jupyter Notebooks for a mix of text and code.

A Python program is a text file containing instructions. You can run a script by using a IDE or just simply using the terminal.

```
python script.py
```

or

```
python3 script.py
```

Note

In some systems, you may need to use `python` instead of `python3`. This situation is related to the installation process of Python that in environment path. It can be fixed by changing the environment path.

As today, the most recent version of Python is 3.12. Most of the Python libraries are dependent on the version of Python. So sometimes, you may need to check your Python compatibility with the libraries you want to use. To check your Python version, you can use the following command:

```
python --version
```

This method usully is a tool to check is your Python works properly. If you need to upgrade:

```
python -m pip install --upgrade python
```

Contary, in some situations, you may need to downgrade. In that case, you can use the following command, however this is not recommended. Instead of downgrading, using virtual environments is a better approach. We will discuss this later, when we deal with OpenSeesPy.

```
python -m pip install python==3.11
```

Let's look at a famous "Hello World" example.

Example (Hello World).

This program prints a message to the screen.

```
print("Hello, Civil Engineering Students!")
print("Welcome to Computer Programming course.")
```

Here is a simple engineering calculation example:

Example (Foundation Area).

Calculates the area of a rectangular footing.

```
# Calculate the area of a foundation footing
width = 2.5  # meters
length = 3.0 # meters

area = width * length

print("Footing Width:", width, "m")
print("Footing Length:", length, "m")
print("Footing Area:", area, "m^2")

# The difference between a calculator and a programming
# script lies in here. Please notice we didnt use dimensions
# in the calculation. We used variables instead. In advance,
# we can code this script like following:

#read inputs from user
width = float(input("Enter footing width (m): "))
length = float(input("Enter footing length (m): "))

#calculate area
area = width * length

#print results
print("Footing Width:", width, "m")
print("Footing Length:", length, "m")
print("Footing Area:", area, "m^2")
```

As you can see, in this example, we didn't assign any number to the variables. We just used the variables to store the values entered by the user. This is the power of programming.

Note

Notice the **#** symbol. This is used for **comments**. The computer ignores everything after the **#** on that line. Comments are for humans to understand the code!

3 Introduction to Algorithms

3.1 What is an Algorithm?

Before we write a single line of code, we must strictly define the problem we are trying to solve. An **algorithm** is a well-defined, step-by-step procedure or a set of rules to be followed in calculations or other problem-solving operations.

Think of an algorithm like a **cooking recipe** or a **construction method statement**. If you are pouring concrete for a column, you don't just "pour concrete". You follow a specific sequence:

1. Build the formwork.
2. Place the reinforcement bars (rebars).
3. Mix the concrete.
4. Pour the concrete into the formwork.
5. Vibrate the concrete to remove air bubbles.
6. Wait for curing.
7. Remove the formwork.

If you change the order (e.g., pour concrete before placing rebars), the result is a disaster. Programming is exactly the same. The computer is a diligent worker who follows your instructions blindly. If your algorithm (instructions) is wrong, the result will be wrong (Garbage In, Garbage Out).

3.2 Properties of a Good Algorithm

A robust algorithm must satisfy the following criteria:

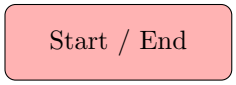
- **Input**: It should accept zero or more inputs.
- **Output**: It should produce at least one output.
- **Definiteness (Unambiguous)**: Each step must be clear and lead to only one meaning. "Mix concrete until it looks good" is bad. "Mix concrete for 5 minutes" is good.
- **Finiteness**: The algorithm must terminate after a finite number of steps. It cannot run forever.
- **Effectiveness**: Every step must be basic enough to be carried out.

3.3 Flowcharts

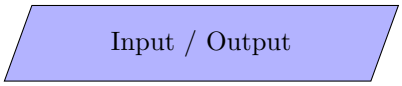
A flowchart is a visual representation of an algorithm. It uses standardized symbols to illustrate the logical flow of the program. Using flowcharts is critical because it allows engineers to discuss the logic without worrying about the specific syntax of a programming language (like Python or C++).

3.3.1 Basic Flowchart Symbols

We use the ISO 5807 standard for flowchart symbols.

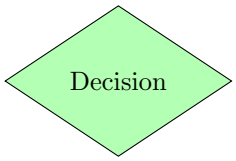


Oval (Terminal): Indicates the beginning or the end of a program flow.

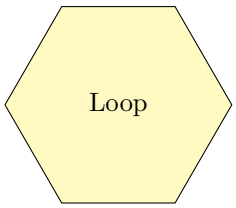


Parallelogram (Input/Output): Denotes asking the user for information (Input) or displaying results to the screen (Output). Example: "Read Beam Length", "Print Area".

Rectangle (Process): Represents a calculation, arithmetic operation, or data assignment. Example: $Area = Width \times Height$.



Diamond (Decision): A branching point. It checks a condition (True/-False or Yes/No) and splits the flow. Example: "Is $Factor\_of\_Safety > 1.5$?".



Hexagon (Preparation): Represents an initialization of a loop. Example: "For i = 1 to 10".

3.3.2 Tools for Flowcharts

While we sketch on paper during brainstorming, professional documentation requires digital tools:

- **Draw.io (diagrams.net)**: A free, open-source, web-based tool. Highly recommended.
- **Microsoft Visio**: Industry standard for corporations.
- **Lucidchart**: A popular web-based alternative.

Example (Algorithm: Concrete Strength Check)**.** Let's design a simple algorithm to check if a concrete sample passes a strength test. **Goal**: Check if measured strength (f_{ck}) is greater than required strength (f_{req}).

Flowchart Logic:

1. **Start** the program.
2. **Input** the measured strength and required strength.
3. **Decision**: Is Measured > Required?

4. **Process (Yes)**: Set status to "PASSED".
5. **Process (No)**: Set status to "FAILED".
6. **Output** the status.
7. **End**.

4 Variables in Programming

4.1 Why we need variables?

Imagine a solver for a structural system. You have thousands of beams, each with a length, modulus of elasticity (E), and moment of inertia (I). If you write code like:

```
stiffness = 3 * 210000 * 0.00045 / (4000 * 4000 * 4000)
```

It is impossible to understand what these numbers mean. Is 210000 the modulus E? Or is it something else? Also, if you need to change E to 200000, you have to find and replace every instance of 210000 manually. This is error-prone.

A **variable** is a symbolic name associated with a value and whose associated value may be changed. It is a container (or a labelled box) in the computer's memory.

4.2 Rules for Naming Variables

Naming is harder than it looks. In Python, we follow these rules and conventions:

1. **Must start with a letter** or underscore (_). Cannot start with a number. (e.g., `1beam` is wrong, `beam1` is correct).
2. **Case Sensitive**: `Area` and `area` are different variables.
3. **No Spaces**: Use underscores (snake_case) to separate words. (e.g., `beam_length`, not `beam length`).
4. **Meaningful Names**: Use `elastic_modulus` instead of `e` or `x`.
5. **Reserved Keywords**: Cannot use Python keywords as variable names. (e.g., `if`, `else`, `for`, `while`, `def`, `class`, etc.). In addition to these, stay away from using most of the standart words as variable names. If you need to use a word that can be used commonly, you can use underscores or any other characters to make it unique. For example, you can use `area_1` instead of `area` even there wont be an `area2`.

Note

Assignment Operator (=): In mathematics, = means logical equality ($x = y$ means x is equal to y). In programming, = is an **action**. `x = 5` means "Take the value 5 and put it into the box named x". `x = x + 1` is mathematically impossible, but in programming, it means "Take the current value of x, add 1, and save the result back into x".

4.3 Variables and Memory Management

To understand variables deeply, we must briefly look inside the computer hardware. When you write `x = 5`, what actually happens inside the machine?

4.3.1 Volatile vs Non-Volatile Memory

A computer has two main types of data storage:

1. **Hard Drive / SSD (Non-Volatile)**: This is your "Bookshelf". It stores your files, photos, and code scripts permanently. Data remains even when power is off.
2. **RAM (Random Access Memory) (Volatile)**: This is your "Desk". When you run a program, the computer loads the data from the hard drive to the RAM. Variables live here.

Note

Volatility: If your computer acts up and you restart it, all variables in the RAM are lost. This is why you must "save" files (write them from RAM to Disk) to keep them.

4.3.2 How Variables are Stored (Bits and Bytes)

Computers run on electricity. A switch is either ON or OFF. We represent this as 1 or 0.

- **Bit**: The smallest unit (0 or 1).
- **Byte**: A group of 8 bits (e.g., `01000001`).

  Everything is a number to a computer.
- An **Integer** '5' is stored in binary as '...000101'.
- A **String** '"A"' is stored using a code (ASCII), where 'A' corresponds to number 65, which is then stored as binary '01000001'.

4.4 Standard and Engineering Data Types

We have covered the basics, but Engineering requires more.

1. **Integer (`int`)**: Exact whole numbers.
2. **Floating Point (`float`)**: Approximations of real numbers.
3. **Complex Numbers (`complex`)**: Numbers with real and imaginary parts ($z = a + bi$).
   - Engineering Application: Essential for **Structural Dynamics**, **Earthquake Engineering** (Frequency Domain analysis), and Electrical circuits.
   - Python Syntax: Use `j` instead of i. Example: `z = 3 + 4j`.
4. **String (`str`)**: Text data.
5. **Boolean (`bool`)**: Logic state (True/False). Efficient storage (can theoretically be 1 bit, though usually stored as a byte).
6. **NoneType (`None`)**: Represents "No Value". Useful for initializing variables that will be calculated later.

Example (Variables including Complex Numbers).

This script demonstrates extended data types including complex numbers.

```
# Variables and Data Types Example

# Integer (Whole number)
number_of_floors = 5
print("Floors:", number_of_floors, "| Type:", type(number_of_floors))

# Float (Decimal number)
beam_length = 4.5
print("Beam Length:", beam_length, "m", "| Type:", type(beam_length))

# Complex (Real + Imaginary)
# Useful for dynamics/vibrations applications
impedance = 3 + 4j
```

```
print("Impedance:", impedance, "| Type:", type(impedance))
print("Real part:", impedance.real, "Imaginary part:", impedance.imag)

# String (Text)
material_name = "Concrete (C30)"
print("Material:", material_name, "| Type:", type(material_name))

# Boolean (True/False)
is_safe = True
print("Is Structure Safe?", is_safe, "| Type:", type(is_safe))

# NoneType (Empty/Null)
# Used to represent 'no value' or initialization
result = None
print("Current result:", result, "| Type:", type(result))
```

4.5 Efficiency and Data Types

Choosing the right data type is critical for **Performance** and **Accuracy**.

4.5.1 Memory Footprint

Different types consume different amounts of RAM.

- Integers are generally compact.
- Floats require more detail (mantissa + exponent) to store the position of the decimal point.
- Strings are "expensive". Storing the number `123` as an integer takes a few bytes. Storing `"123"` as a string takes significantly more space because each character ('1', '2', '3') has its own code, plus overhead data.

4.5.2 Computational Speed

CPUs can process integers extremely fast. Floating-point operations take more cycles. **Optimization Tip**: If you are counting items (e.g., iterations, objects), always use Integers. Use Floats only when you need decimal precision.

4.5.3 Precision vs Exactness

- **Integers are Exact**: $1 + 1 = 2$. Always.
- **Floats are Approximate**: Computers cannot represent infinite precision (like $1/3 = 0.3333...$). Sometimes, 0.1+0.2 might result in 0.30000000000000004 due to binary floating-point arithmetic.
- **Engineering Rule**: Never compare floats for exact equality. Do not check if `stress == 20.0`. Instead, check if it is close enough: `abs(stress - 20.0) < 0.0001`.

4.6 Type Conversion (Casting)

Since variables have types, we sometimes need to change them. This is called **casting**. The most common scenario in engineering scripting is reading input. The `input()` function always returns a **string**, even if the user types a number.

```
age = input("Enter age: ") # User types 20
# age is now "20" (String)
# next_year = age + 1  <-- ERROR! String + Number
```

We must cast it:

```
age_num = int(age) # Converts "20" to 20 (Integer)
next_year = age_num + 1 # Works!
```

Example (Type Conversion).

```
# Type Conversion (Casting) Example

# String to Float
displacement_str = "12.5"
displacement_val = float(displacement_str)
print("Displacement (float):", displacement_val)

# Float to Integer
# Note: This truncates the decimal part!
load = 15.8
load_int = int(load)
print("Load (original):", load)
print("Load (int):", load_int)

# Number to String
# Useful for combining with other text
force = 100
message = "The applied force is " + str(force) + " kN"
print(message)
```

Question 1

What happens if you try to convert the string `"2.5"` into an integer using `int("2.5")`? *Hint: Integers must be whole numbers.*

Exercise 4.1. Create a flowchart for engineering problem that you are familiar with.

4.7 Introduction to Functions

The function concept and notaion will be handled in future weeks, however, in learning phase we'll use some functions already. So this section is just a brief introduction to functions. As engineers, we love efficiency. We don't want to rewrite the same formula for "Area of a Circle" or "Moment of Inertia" fifty times in our code. A **Function** is a block of code which only runs when it is called. You can think of it as a "mini-program" inside your main program.

- You **define** it once.
- You **call** (use) it as many times as you like.

We interpret functions as a "Black Box":

- **Input** (Arguments): Raw materials go in.
- **Processing**: The function does some work.
- **Output** (Return): The finished product comes out.

Example (A Simple Function).

```
# Introduction to Functions
# ------------------------
# A function is a reusable block of code.
# We define it once, and 'call' it many times.

# 1. Defining a simple function
def calculate_area(width, height):
    """Calculates area of a rectangle"""
    area = width * height
    return area

# 2. Using (Calling) the function
b1 = 3.0
h1 = 4.0
area1 = calculate_area(b1, h1)
print(f"Beam 1 (3x4) Area: {area1}")

b2 = 5.0
h2 = 0.5
area2 = calculate_area(b2, h2)
print(f"Beam 2 (5x0.5) Area: {area2}")

# 3. Built-in Functions we already use
# print(), type(), input(), int(), float() are all functions provided by Python.
print("Length of string 'Concrete':", len("Concrete"))
```

Note

We will examine Functions in detail. For now, just understand that commands like `print()`, `type()`, and `input()` are actually built-in functions provided by Python.

5 Operators and Expressions

In the previous week, we learned about **variables** (the nouns of programming). Now, we will learn about **operators** (the verbs). Operators allow us to manipulate variables to perform data processing, calculations, and logic making.

An **Expression** is a combination of variables and operators that yields a result. For example, in the expression `a + b`:

- `a` and `b` are **operands**.
- `+` is the **operator**.

5.1 Arithmetical operators

These are the most familiar operators, used for mathematical calculations. In Civil Engineering, these are the bread and butter of your design scripts (e.g., calculating areas, moments, stiffness matrices).

Symbol	Name	Example	Description
`+`	Addition	`x + y`	Adds two numbers
`-`	Subtraction	`x - y`	Subtracts right from left
`*`	Multiplication	`x * y`	Multiplies two numbers
`/`	Division	`x / y`	Divides (Always returns float)
`%`	Modulus	`x % y`	Returns the remainder
`**`	Exponentiation	`x ** y`	x^y (Power)
`//`	Floor Division	`x // y`	Division result truncated to integer

Table 1: Python Arithmetic Operators

Note

Syntax Detail: In some older languages or low-level languages (like C), dividing two integers (1/2) results in 0 (integer). In Python 3 (High-level), `1/2` results in `0.5` (float), which is more intuitive for engineers. However, strict type handling is crucial in software like SAP2000 APIs.

Example (Calculations).

We calculate momentum, and demonstrate modulus usage for batching.

```
# Arithmetic Operators
# ---------------------
L = 5.0    # Beam Length (m)
w = 12.0   # Distributed Load (kN/m)

# Exponentiation (**): M = w * L^2 / 8
moment = (w * L**2) / 8
print("Maximum Moment:", moment, "kNm")

# Modulus (%): Useful for cyclic patterns or checking remainders
# Example: We have 52 rebars, want to bundle them in groups of 10.
```

```
total_rebars = 52
group_size = 10
remaining = total_rebars % group_size
print("Rebars left over:", remaining)

# Floor Division (//): Integer result of division
filled_groups = total_rebars // group_size
print("Full groups created:", filled_groups)


# Assignment Operators
# ----------------------
# Instead of writing x = x + 1, we write x += 1
concrete_volume = 100 # m3
print("Initial Volume:", concrete_volume)

# Pour 20 m3 more
concrete_volume += 20
print("After Pour:", concrete_volume)

# A mixer truck takes 8 m3, remove it
concrete_volume -= 8
print("After Truck Load:", concrete_volume)
```

5.2 Assignment operators

As discussed last week, `=` is an assignment, not equality. We often want to modify a variable's existing value. Instead of writing `x = x + 5`, we can use the shorthand `x += 5`. This helps usually in performance in low-level languages (not much in Python) but mostly in readability.

- `+=` : Add and assign (`a += b` → `a = a + b`)
- `-=` : Subtract and assign
- `*=` : Multiply and assign

5.3 Comparison operators

These operators compare two values and evaluate to a **Boolean** value (`True` or `False`). This is the heart of engineering decision making. "Is $Demand < Capacity$?" "Is $Stress > YieldStrength$?"

Symbol	Name	Engineering Context
`==`	Equal	Is span length exactly 5.0m?
`!=`	Not Equal	Is material different from Steel?
`>`	Greater Than	Is Load > Capacity? (Failure?)
`<`	Less Than	Is Deflection < Limit? (Safe?)
`>=`	Greater/Equal	Is Factor of Safety ≥ 1.5?
`<=`	Less/Equal	Is cost ≤ budget?

Note

Common Mistake: Do not confuse `=` (Assignment) with `==` (Comparison). `if x = 5` is a syntax error in Python. `if x == 5` is a question.

5.4 Logical operators

In real projects, decisions rely on multiple conditions. "The design is acceptable IF (Safety Check is OK) **AND** (Cost is Low)."

- **and**: Returns True if **both** operands are true.
- **or**: Returns True if **at least one** operand is true.
- **not**: Reverses the result (True becomes False).

Example (Design Checks).

Combining stress limits and deflection limits.

```
# Comparison Operators
# ---------------------
# Comparing Demand vs Capacity is fundamental in Civil Engineering
demand_load = 150.0  # kN
capacity = 200.0     # kN

is_safe = capacity > demand
print("Is the structure safe?", is_safe)
print("Is critical?", capacity == demand)


# Logical Operators (and, or, not)
# ---------------------------------
# Complex conditions often require multiple checks.

# Example: A generic design check
# 1. Stress must be below limit
# 2. Deflection must be below limit

stress = 150 # MPa
deflection = 12 # mm

stress_limit = 250 # MPa
deflection_limit = 20 # mm

# AND: Both must be True
design_check = (stress < stress_limit) and (deflection < deflection_limit)
print("Design passes checks:", design_check)

# OR: At least one is True
# Example: Warning if ANY limit is approached (say 80% usage)
warning_needed = (stress > 0.8 * stress_limit) or (deflection > 0.8 *
    deflection_limit)
print("Warning light on:", warning_needed)

# NOT: Reverses the boolean
print("Design failed:", not design_check)
```

5.5 Bitwise operators

Bitwise operators treat numbers as binary strings (bits) rather than decimal numbers. They operate bit by bit.

- `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (Left Shift), `>>` (Right Shift).

While rarely used in high-level structural analysis scripts, they are critical in:

1. **Optimization**: Checking flags in FEA software (Ansys used bitmasks for node selection sets).
2. **Data Compression**: Storing massive point cloud data (LiDAR).
3. **Low-level Hardware**: Reading data from sensors (Arduino/Raspberry Pi) for Structural Health Monitoring.

Question 2

If `a = 10` (Binary: 1010) and `b = 4` (Binary: 0100), what is `a & b`? *Hint: Compare bits in each position. 1 AND 0 is 0.*

Exercise 5.1 (Beam Safety Verification). **Objective**: Write a Python script to verify the safety of a simply supported steel beam under a distributed load.

Given Parameters:

- Span Length (L): 6.0 m
- Distributed Load (w): 15.0 kN/m
- Elastic Modulus (E): 210×10^6 kPa (kN/m^2)
- Moment of Inertia (I): 0.0002 m^4
- Section Modulus (Z): 0.001 m^3
- Yield Strength (F_y): 250×10^3 kPa

Calculations:

1. Maximum Moment: $M_{max} = \frac{wL^2}{8}$
2. Bending Stress: $\sigma = \frac{M_{max}}{Z}$
3. Max Deflection: $\delta_{max} = \frac{5wL^4}{384EI}$
4. Allowable Deflection: $\delta_{allow} = \frac{L}{300}$

Pass Condition: The design is SAFE if **both** conditions are met:

1. $\sigma \leq F_y$
2. $\delta_{max} \leq \delta_{allow}$

Task: Implement variables, arithmetic operators for formulas, comparison operators for checks, and logical operators for the final decision. Print the final boolean result.

```
# Week 3 Exercise: Beam Safety Verification

# Input Data
# ----------------------------------------
```

```
L = 6.0          # Beam Length (m)
w = 15.0         # Distributed Load (kN/m)
E = 210000000.0 # Elastic Modulus (kPa or kN/m^2) [Steel]
I = 0.0002       # Moment of Inertia (m^4)
Z = 0.001        # Section Modulus (m^3)
Fy = 250000.0    # Yield Strength (kPa)

# 1. Calculate Maximum Moment (kNm)
# Formula: M = w * L^2 / 8
M_max = (w * L**2) / 8
print("Maximum Moment:", M_max, "kNm")

# 2. Calculate Bending Stress (kPa)
# Formula: Stress = M / Z
sigma = M_max / Z
print("Bending Stress:", sigma, "kPa")

# 3. Calculate Maximum Deflection (m)
# Formula: delta = 5 * w * L^4 / (384 * E * I)
delta_max = (5 * w * L**4) / (384 * E * I)
print("Max Deflection:", delta_max * 1000, "mm")

# 4. Define Limits
allowable_deflection = L / 300
print("Allowable Deflection:", allowable_deflection * 1000, "mm")

# 5. Perform Safety Checks
# Check 1: Is Stress within Elastic Range? (Stress <= Yield)
check_stress = sigma <= Fy

# Check 2: Is Deflection acceptable? (Delta <= Limit)
check_deflection = delta_max <= allowable_deflection

# Final Decision: BOTH must be True for the design to be Safe
is_design_safe = check_stress and check_deflection

print("-" * 30)
print("Stress Check:", check_stress)
print("Deflection Check:", check_deflection)
print("IS DESIGN SAFE?", is_design_safe)
print("-" * 30)
```

6 Control Flow - Part 1 : If-Else Statements

Up until now, our programs have been linear. The computer executes line 1, then line 2, then line 3, and so on. However, real life is not linear. We make decisions based on conditions.

- "If it rains, take an umbrella."
- "If $\sigma > F_y$, the beam fails."
- "If concrete strength is C30, use $E = 33$ GPa."

Control Flow allows us to change the order of execution based on logic. The most fundamental tool for this is the **Conditional Statement**.

6.1 Conditional Statements

Conditional statements allow the program to execute a block of code **only if** a specific condition is true.

6.1.1 If-Else Statements

In Python, the syntax relies heavily on **Indentation** (whitespace). Unlike C++ or Java which use curly braces {}, Python uses the colon : and indentation to define blocks.

Structure:

```
if condition:
    # Code to run if True
    # (Indented block)
elif another_condition:
    # Code to run if first is False AND this is True
else:
    # Code to run if ALL above are False
```

Example (Concrete Strength Classification).

Classification of concrete based on compressive strength.

```
# Conditional Statements (if-elif-else)
# -----------------------------------------
# Example: Determines the strength class of a concrete sample
# based on its characteristic cylinder strength (f_ck).

f_ck = float(input("Enter the characteristic cylinder strength (f_ck) in MPa: "))

# Logic:
# - Below 20 MPa: Low Strength (Non-structural usage mostly)
# - 20 to 50 MPa: Normal Strength (Standard buildings)
# - 50 to 90 MPa: High Strength (High-rise, Bridges)
# - Above 90 MPa: Ultra-High Strength

if f_ck < 20:
    category = "Low Strength Concrete"
    application = "Pavements, blinding concrete"
elif f_ck < 50:
    category = "Normal Strength Concrete"
    application = "Beams, columns, slabs in typical buildings"
elif f_ck < 90:
    category = "High Strength Concrete"
```

```
    application = "High-rise buildings, long-span bridges"
else:
    # If none of the above are true
    category = "Ultra-High Strength Concrete"
    application = "Specialized structural elements, nuclear containment"

print("-" * 30)
print("Classification:", category)
print("Typical Use:", application)
print("-" * 30)
```

6.1.2 Switch Statements (Match-Case)

Historically, Python did not have a `switch` statement like C or Java. Programmers used long chains of `if-elif-else` blocks. However, Python 3.10 introduced the `match-case` statement, which is a powerful structural pattern matching tool.

It is best used when checking a single variable against specific distinct values (e.g., Menu options, Material codes, Load cases).

Example (Load Factors).

Selecting Partial Safety Factors based on Load Type codes.

```
# Switch Statements (match-case)
# ------------------------------
# Introduced in Python 3.10, 'match-case' is the powerful equivalent
# of 'switch' statements in C/C++/Java.
# It is used when comparing a variable against a list of specific values.

# Example: Get the Partial Safety Factor (gamma) for different load types
# based on Eurocode / Local Standards.

load_type = input("Enter the load type (D, L, W, E): ")

gamma = 0.0
description = ""

match load_type:
    case "D":
        description = "Dead Load"
        gamma = 1.4 # or 1.35 per EC
    case "L":
        description = "Live Load"
        gamma = 1.6 # or 1.5 per EC
    case "W":
        description = "Wind Load"
        gamma = 1.0 # factor varies by combination, simplified here
    case "E":
        description = "Earthquake Load"
        gamma = 1.0
    case _:
        # The underscore (_) acts as a default/catch-all case
        description = "Unknown Load Type"
        gamma = 1.0

print("-" * 30)
print(f"Load: {description}")
print(f"Partial Safety Factor (gamma): {gamma}")
```

```
print("-" * 30)
```

Note

Indentation Matters: In Python, indentation is not just for style; it is syntax.

```
if x > 5:
print("Big") # ERROR! IndentationError
```

You must indent (typically 4 spaces) the code inside the block.

Exercise 6.1 (Fluid Flow Regimes). **Objective**: Determine the flow regime of a fluid in a pipe based on the Reynolds Number (Re).

Theory: The Reynolds number is a dimensionless quantity used to predict flow patterns.

1. **Laminar Flow**: $Re < 2000$. Smooth, constant flow.
2. **Transitional Flow**: $2000 \leq Re < 4000$. Unstable flow.
3. **Turbulent Flow**: $Re \geq 4000$. Chaotic flow.

Task: Write a Python script that:

1. Defines a variable `Re` (e.g., 3500).
2. Uses `if-elif-else` logic to determine the regime.
3. Prints the result in a sentence: "For Re = 3500, the flow is Transitional."

7 Control Flow - Part 2: Loops

In engineering, we rarely do a calculation just just once. We typically want to:

- Analyze **all** beams in a floor.
- Find the root of an equation by **iterating** until error is small.
- Integrate a complex function by summing **many** small areas.

Loops allows us to repeat a block of code multiple times.

7.1 For Loops

The `for` loop is used when we know the number of iterations in advance, or when we want to iterate over a sequence (like a list of items).

Syntax:

```
for variable in sequence:
    # Code to repeat
```

7.1.1 The `range()` function

Ideally suited for generating sequences of numbers.

- `range(5)`: Generates 0, 1, 2, 3, 4.
- `range(2, 6)`: Generates 2, 3, 4, 5.
- `range(0, 10, 2)`: Generates 0, 2, 4, 6, 8 (Step size 2).

Example (Iterating over Lists and Ranges).

Summing point loads and calculating cantilever moments.

```
# For Loops
# ----------
# Iterate over a known sequence or range.

# Example 1: Summing loads acting on a beam
point_loads = [10.5, 20.0, 5.0, 15.0] # kN
total_load = 0.0

print("Calculating Total Load...")
for load in point_loads:
    print(f"Adding load: {load} kN")
    total_load += load

print(f"Total Point Load: {total_load} kN")
print("-" * 30)

# Example 2: Iterating with an index using range()
# Calculating moment at specific intervals
# Moment M(x) = w * x^2 / 2 (Cantilever under dist. load)
w = 10.0 # kN/m
length = 5.0 # m
steps = 6 # 0, 1, 2, 3, 4, 5

print("Moment Distribution along Cantilever:")
```

```
for x in range(steps):
    moment = (w * x**2) / 2
    print(f"Distance x={x}m -> Moment={moment} kNm")
```

7.2 While Loops

The `while` loop is used when we do not know how many times we need to loop. It continues **as long as** a condition remains `True`.

Syntax:

```
while condition:
    # Code to repeat
```

Note

Infinite Loops: If the condition never becomes `False`, the program will run forever (or crash). You must ensure that something inside the loop changes the condition eventually.

Example (Iterative Design).

Finding the minimum required concrete grade by iteratively increasing it until capacity meets demand.

```
# While Loops
# ----------
# Repeat code WHILE a condition is true.
# Useful when we don't know how many iterations we need beforehand.

# Example: Designing a column section (Iterative Approach)
# We strictly increase the concrete grade until capacity > demand.

demand = 2500 # kN
capacity = 0 # Initial
grade = 20 # Start with C20 concrete
area = 0.4 * 0.4 # 400x400 mm column

print(f"Required Capacity: {demand} kN")

while capacity < demand:
    # Calculate capacity (Simplified: 0.85 * fck * Area)
    # We ignore steel for this simple logic example
    fck = grade * 1000 # kPa
    capacity = 0.85 * fck * area

    print(f"Trying C{grade}: Capacity = {capacity:.1f} kN")

    if capacity < demand:
        grade += 5 # Increase grade by 5 MPa (C20 -> C25 -> C30...)

print("-" * 30)
print(f"Design Selected: C{grade} with Capacity {capacity:.1f} kN")
```

7.3 Do-While Loops

Many languages (C, C++, Java) have a `do-while` loop, which guarantees that the code runs **at least once** before checking the condition. Python **does not** have a built-in `do-while` statement. However, we can emulate it:

```
while True:
    # Code runs here
    if condition_to_stop:
        break # Exit the loop manually
```

7.4 Loop Control Statements

- **break**: Terminates the loop immediately.
- **continue**: Skips the rest of the current iteration and jumps to the next one.

Exercise 7.1 (Numerical Integration). **Objective**: Calculate the total force acting on a beam under a distributed load defined by a function $w(x)$, using numerical integration.

Theory: The total force F is the area under the load curve $w(x)$.

$$F = \int_0^L w(x)\,dx$$

Computers cannot do symbolic calculus easily, but they can do very fast addition. We can approximate the integral by dividing the area into N small rectangles of width dx.

$$F \approx \sum_{i=0}^{N-1} w(x_i) \cdot dx$$

Task: 1. Define a load function for a trapezoidal load: $w(x) = 2x + 5$ (kN/m). 2. Input the beam length L and number of steps N. 3. Write a loop to sum the areas $w(x_i) \cdot dx$. 4. Compare with the exact mathematical solution.

```
# Exercise: Numerical Integration (Riemann Sum)
# ---------------------------------------------
# Calculate the integral of a function f(x) from a to b.
# Engineering Context: Total force from a varying distributed load.
# Load Function: w(x) = 2*x + 5 (Trapezoidal load distribution)
# Integration Range: x = 0 to x = L

def w(x):
    """Distributed load function w(x) = 2x + 5"""
    return 2.0 * x + 5.0

# Integration Parameters
print("Calculating Total Force from Distributed Load w(x) = 2x + 5")

L = float(input("Enter beam length L (m): ")) # Integration upper limit (b)
N = int(input("Enter number of segments N: "))  # Number of steps

# Step size
dx = L / N
```

```
total_force = 0.0

# Numerical Integration using Midpoint Rule (or simple Left/Right sum)
# Integral ~ Sum( w(x_i) * dx )
print(f"integrating from 0 to {L} with {N} steps (dx={dx})...")

for i in range(N):
    # Calculate x at the midpoint of the current slice for better accuracy
    x_i = i * dx + (dx / 2)

    # Calculate area of this slice
    force_slice = w(x_i) * dx

    # Add to total
    total_force += force_slice

print("-" * 30)
print(f"Total Force (Approx): {total_force:.4f} kN")

# Exact Analytic Solution: Integral(2x + 5) = x^2 + 5x
# F_exact = (L^2 + 5*L) - (0)
exact_force = (L**2) + 5*L
print(f"Total Force (Exact) : {exact_force:.4f} kN")
print(f"Error               : {abs(total_force - exact_force):.6f} kN")
print("-" * 30)
```

8 Data Structures and Libraries

Engineering problems typically involve huge sets of data, not just single variables.

- A bridge has hundreds of beams. We cannot create `beam1`, `beam2`, ..., `beam100`.
- A material has many properties (E, ν, ρ, f_y). We need to group them.

One of the most powerful features of Python is its ability to hande these kind of datasets. These are called **Data Structures**. For different purposes, there are different data structures. The most common ones are lists, tuples, and dictionaries. The problem is different libraries use different data structures. For example, NumPy uses arrays, Pandas uses dataframes, etc. If the developer is aware of the kind of data structure to use, easily can convert between them.

8.1 Lists

A **List** is an ordered, changeable collection of items. It is the most versatile data type in Python. You can store numbers, strings, or even other lists inside a list.

Syntax:

```
my_list = [item1, item2, item3]
```

8.1.1 Indexing and Slicing

We access list items using their index. Remember, Python (and many other programming languages) starts counting at **0**. If you are a beginner in programming, this might be confusing however it is actually make sense. The reason of starting from 0 is related with the memory allocation of arrays.

- `loads[0]`: The first item.
- `loads[-1]`: The last item. (Not recommended, instead use `loads[len(loads)-1]`)
- `loads[0:3]`: **Slicing**. Gets items from index 0 up to (but not including) index 3.

Example (List Operations).

Managing structural loads using lists: Indexing, Modifying, and Calculating Totals.

```python
# Python Lists
# ------------
# Lists are ordered collections of items.
# Engineering Use: Storing multiple load values, beam lengths, or sensor readings.

# 1. Defining a List
# ------------------
# Loads in kN acting on a structure
loads = [10.5, 25.0, 5.0, 15.0, 0.0]
print(f"All Loads: {loads}")

# 2. Accessing Elements (Indexing)
# --------------------------------
# Python is 0-indexed.
first_load = loads[0]
last_load = loads[-1] # Negative index counts from the end
print(f"First Load: {first_load} kN")
print(f"Last Load: {last_load} kN")
```

```
# 3. Slicing (Subsets)
# ------------------------
# We want to analyze only the first 3 loads
# Syntax: list[start:end] (end is exclusive)
subset = loads[0:3]
print(f"First 3 loads: {subset}")

# 4. Modifying Lists (Mutable)
# ------------------------------
# Adding a new load (e.g. from a new user input)
loads.append(30.0)
print(f"After Append: {loads}")

# Correcting a value (e.g. data correction)
loads[2] = 7.5
print(f"After Correction: {loads}")

# 5. Engineering Operation
# ------------------------------
# Calculate Total Load and Max Load
total_load = sum(loads)
max_load = max(loads)

print("-" * 30)
print(f"Total Design Load: {total_load} kN")
print(f"Critical (Max) Load: {max_load} kN")
print("-" * 30)

# 6. Converting Data Structures
# ------------------------------
# Converting a List to a Tuple (to make it immutable/read-only)
# This is useful when you want to ensure data cannot be changed accidentally
loads_tuple = tuple(loads)
print(f"Loads as Tuple: {loads_tuple}")

# Converting a Tuple back to a List (to make edits)
# If we need to modify data again, we convert it back to a list
loads_list_again = list(loads_tuple)
loads_list_again.append(50.0)
print(f"Loads back as List (with new item): {loads_list_again}")
```

8.2 Dictionaries

A **Dictionary** is a collection which is unordered, changeable, and indexed. In Python dictionaries are written with curly brackets {}, and they have **keys** and **values**.

This is perfect for Engineering Properties. Instead of remembering that "index 0 is Elastic Modulus", we can access it using `material["E"]`.

```
beam_props = {
    "length": 5.0,
    "material": "Steel",
    "I": 0.00045
}
```

Example (Material Database).

Defining material properties (Concrete, Steel) using Dictionaries and calculating member

weight.

```
# Python Dictionaries
# ------------------
# Dictionaries store data in Key-Value pairs.
# Engineering Use: Storing properties of defined materials or sections.
# Unlike lists, data is accessed by "Name" (Key), not index.

# 1. Defining a Material Database
# ------------------------------
concrete_C30 = {
    "type": "Concrete",
    "grade": "C30",
    "fck": 30.0,        # MPa
    "E": 33000.0,       # MPa (Elastic Modulus)
    "gamma": 25.0,      # kN/m3 (Unit Weight)
    "is_ductile": False
}

steel_S420 = {
    "type": "Steel",
    "grade": "S420",
    "fy": 420.0,        # MPa
    "E": 200000.0,      # MPa
    "gamma": 78.5,      # kN/m3
    "is_ductile": True
}

# 2. Accessing Data
# -----------------
print(f"Analyzing {concrete_C30['type']} {concrete_C30['grade']}...")
print(f"Characteristic Strength: {concrete_C30['fck']} MPa")

# 3. Modifying Data
# -----------------
# Updating a property (e.g. after lab test)
concrete_C30["fck"] = 32.5
print(f"Updated fck: {concrete_C30['fck']} MPa")

# Adding a new property
concrete_C30["cost_per_m3"] = 1500.0 # TL
print("Added Cost info:", concrete_C30)

# 4. Application: Calculating Weight of a Column
# ----------------------------------------------
# Column: 300x300 mm, Height: 3 m
b = 0.3 # m
h = 0.3 # m
L = 3.0 # m
volume = b * h * L

# Weight = Volume * Unit Weight (gamma)
weight = volume * concrete_C30["gamma"]

print("-" * 30)
print(f"Column Volume: {volume:.3f} m3")
print(f"Column Weight: {weight:.3f} kN using {concrete_C30['grade']} concrete.")
print("-" * 30)
```

8.3 Tuples

A **Tuple** is a collection which is ordered and **unchangeable** (immutable). We use tuples for data that should not change during the program, like coordinates of a fixed node.

```
node_01 = (0.0, 5.0, 3.5) # x, y, z coordinates
# node_01[0] = 2.0  <— ERROR! Cannot change a tuple.
```

8.4 Introduction to Libraries

Python by itself is simple. Its true power comes from the ecosystem of `Libraries` (or Packages). A library is a collection of pre-written functions and classes that you can `import` and use. The open-source community has developed a vast ecosystem of libraries for Python, making it a versatile language for many applications. For almost every specific purpose, there is a library for it, and they are getting much smoother to use day by day.

Note

Until this point, we didn't mention about the classes properly. The class is an important concept in programming. As can be understood from the name, it is a general classification structure (Actually, the word of structure is a bit misleading. Because "structure" is also a different concept). Classes are very helpful to organize the functions, variables, and other data structures in a logical way. In the following weeks, we will discuss about the classes a little bit more.

8.4.1 Why use libraries?

Unless you cannot find a problem that has not been solved yet, you do not need to write your own library (unless you are creating something completely new). You can think of libraries as a toolbox. In that toolbox, you can find different tools (functions, classes, etc.) for different purposes. The only thing you need to know is which tool to use for which purpose (you cannot use a hammer to drive a screw — not properly, at least).

When you get familiar with programming, there will be an instinct to write your own functions, classes, or data structures. However, this instinct should be controlled. Especially in big projects that are developed by teams, it is very important to use existing solutions. Writing code is one thing, but writing it efficiently is another. Most commonly used libraries are developed by experts in the field and optimized for performance. You should spend your time understanding the content of libraries. This is one of the most efficient ways to improve yourself as a programmer.

- **Efficiency**: Don't reinvent the wheel. Someone has already written a highly optimized matrix solver (NumPy).
- **Capabilities**: Python doesn't built-in support for plotting graphs. We need a library (Matplotlib) for that.

8.4.2 The Python Package Index (PyPI) and pip

`PyPI` (pypi.org) is the repository for Python software. `pip` is the package installer for Python.

```
# Installing a library via terminal
pip install numpy
pip install matplotlib
```

8.4.3 Commonly Used Engineering Libraries

The Python ecosystem is vast, containing thousands of libraries. For engineers, these libraries serve as the primary toolkit for solving complex problems ranging from numerical analysis to data visualization and machine learning. Here are some of the most essential ones:

NumPy (Numerical Python)

The cornerstone of scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently.

- **Capabilities**: High-performance multidimensional array object, tools for integrating C/C++ and Fortran code, linear algebra, Fourier transform, and random number capabilities.
- **Key Functions**: `np.array()`, `np.linspace()`, `np.linalg.solve()`, `np.fft.fft()`, `np.random.rand()`.

Pandas

A fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool. It is built on top of NumPy and is essential for handling structured data.

- **Capabilities**: Data manipulation and analysis, DataFrame object for data manipulation with integrated indexing, reading and writing data between different formats (CSV, Excel, SQL, JSON).
- **Key Functions**: `pd.read_csv()`, `pd.DataFrame()`, `df.groupby()`, `df.merge()`, `df.plot()`.

Matplotlib

The standard plotting library. It provides an object-oriented API for embedding plots into applications and creates publication-quality figures.

- **Capabilities**: Publication quality figures in a variety of hardcopy formats, comprehensive library for creating static, animated, and interactive visualizations.
- **Key Functions**: `plt.plot()`, `plt.scatter()`, `plt.bar()`, `plt.contourf()`, `plt.savefig()`.

SciPy (Scientific Python)

A collection of mathematical algorithms and convenience functions built on the NumPy extension.

- **Capabilities**: Algorithms for optimization, integration, interpolation, eigenvalue problems, algebraic equations, differential equations, and statistics.
- **Key Functions**: `optimize.minimize()`, `integrate.odeint()`, `interpolate.griddata()`, `signal.butter()`.

SymPy

A Python library for symbolic mathematics. It aims to become a full-featured Computer Algebra System (CAS).

- **Capabilities**: Symbolic mathematics, algebraic simplification, calculus (derivatives involved), solving equations, discrete math, matrices.
- **Key Functions**: `sp.symbols()`, `sp.diff()`, `sp.integrate()`, `sp.solve()`, `sp.simplify()`.

Scikit-learn

The industry standard for Machine Learning in Python. It provides simple and efficient tools for predictive data analysis.

- **Capabilities**: Classification, regression, clustering, dimensionality reduction, model selection, and preprocessing.
- **Key Functions**: `LinearRegression()`, `KMeans()`, `train_test_split()`, `metrics.mean_squared_error()`.

Seaborn

A statistical data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

- **Capabilities**: Dataset-oriented API for examining relationships between multiple variables, specialized support for categorical variables, visualizing univariate and bivariate distributions.
- **Key Functions**: `sns.heatmap()`, `sns.pairplot()`, `sns.boxplot()`, `sns.violinplot()`.

OpenPyXL / XlsxWriter
Libraries for reading and writing Excel 2010 xlsx/xlsm files.

- **Capabilities**: Reading and writing Excel files, programmatic data formatting, cell manipulation, and chart creation within Excel files.
- **Key Functions**: `load_workbook()`, `sheet.cell()`, `workbook.save()`.

Statsmodels
A library that complements SciPy for statistical modeling. It offers classes and functions for the estimation of many different statistical models.

- **Capabilities**: Estimation of statistical models, conducting statistical tests, statistical data exploration, linear regression, time series analysis.
- **Key Functions**: `OLS()`, `tsa.ARIMA()`, `stats.ttest_ind()`, `qqplot()`.

Plotly
A graphing library for making interactive, publication-quality graphs online.

- **Capabilities**: Interactive graphing, creating complex interactive dashboards, 3D charts, statistical graphs, maps.
- **Key Functions**: `px.scatter_3d()`, `go.Figure()`, `px.line()`, `fig.write_html()`.

NetworkX
A tool for the creation, manipulation, and study of the structure, dynamics, and functions of complex networks.

- **Capabilities**: Data structures for graphs, digraphs, and multigraphs, many standard graph algorithms, network structure and analysis measures.
- **Key Functions**: `nx.shortest_path()`, `nx.draw()`, `nx.degree_centrality()`, `nx.minimum_spanning_tree()`

ObsPy
A comprehensive framework for processing seismological data.

- **Capabilities**: Read and write support for many file formats (MSEED, SAC, etc.), signal processing routines, easy access to data centers, seismological signal analysis.
- **Key Functions**: `read()`, `st.filter()`, `st.plot()`, `signal.invsim()`.

TensorFlow / PyTorch `(Advanced)`
The two dominant open-source libraries for Deep Learning.

- **Capabilities**: End-to-end open source platform for machine learning, flexible ecosystem of tools, deep learning models, neural networks.
- **Key Functions**: `tf.keras.layers.Dense()`, `torch.nn.Linear()`, `model.fit()`, `backward()`.

9 Math, Arrays, and Dataframes

In Week 7, we will dive deeper into the core tools that make Python an Engineering Powerhouse. We will move beyond simple lists and basic arithmetic to specialized Libraries designed for heavy mathematical lifting.

9.1 Math with Python

Python has a built-in module called **math** that provides standard mathematical functions. While simple operators (`+`, `-`, `*`, `/`) are useful, they cannot calculate sine, cosine, or square roots.

9.1.1 The `math` Module

To use it, you must first import it:

```
import math
```

1. **Constants**: High-precision values for π (`math.pi`) and e (`math.e`).
2. **Rounding Tricks**:
   - `round(x)`: Standard rounding (to nearest integer).
   - `math.floor(x)`: Rounds **down** to the nearest integer. (e.g., $3.9 \rightarrow 3$).
   - `math.ceil(x)`: Rounds **up** to the nearest integer. (e.g., $3.1 \rightarrow 4$). Used when you need "at least" X items.
3. **Trigonometry**: **CRITICAL WARNING**: All computer trig functions (sin, cos, tan) expect the angle in **RADIANS**, not degrees.
   - To convert: `math.radians(degrees)`.
   - Example: `math.sin(math.radians(90))`.
4. **The "isclose" Trick**: Floating point numbers are not exact. `0.1 + 0.2` is often `0.30000000000000004`. **NEVER** check if `a == b` for floats. **ALWAYS** use `math.isclose(a, b)`.

Example (Math Module Functions).

A comprehensive guide to using math functions, including the differences between floor/ceil and degrees/radians.

```
import math

# 1. Basic Constants
# ------------------
print(f"Pi: {math.pi}")
print(f"Euler's number (e): {math.e}")

# 2. Rounding Functions (The Tricks)
# ----------------------------------
val = 3.6
print(f"\nValue: {val}")
print(f"round(3.6): {round(val)}")        # Standard rounding (nearest even for .5
    in Python 3!)
print(f"math.floor(3.6): {math.floor(val)}") # Always down
print(f"math.ceil(3.6): {math.ceil(val)}")   # Always up
```

```
print(f"math.trunc(3.6): {math.trunc(val)}") # Cuts decimal part

# Trick: Rounding to specific decimal places
# Round 3.14159 to 2 decimal places
pi_approx = round(math.pi, 2)
print(f"Round(pi, 2): {pi_approx}")

# 3. Powers and Roots
# ------------------
print("\nPowers and Roots:")
print(f"2^3 (pow): {math.pow(2, 3)}") # Returns float
print(f"Sqrt(16): {math.sqrt(16)}")

# 4. Logarithms
# -------------
# Natural Log (ln) -> math.log(x)
# Base 10 Log -> math.log10(x)
val = 100
print(f"\nNatural Log (ln) of {val}: {math.log(val):.4f}")
print(f"Log10 of {val}: {math.log10(val)}")

# 5. Trigonometry (Crucial: RADIANS vs DEGREES)
# ---------------------------------------------
angle_deg = 45
angle_rad = math.radians(angle_deg) # Convert deg to rad

print(f"\nAngle: {angle_deg} degrees")
print(f"Angle: {angle_rad:.4f} radians")
print(f"Sin(45 deg): {math.sin(angle_rad):.4f}") # sin expects radians!
print(f"Cos(45 deg): {math.cos(angle_rad):.4f}")

# Inverse Trig
val = 1.0
print(f"Arcsin(1.0): {math.degrees(math.asin(val))} degrees") # Result in radians,
    convert to deg

# 6. Floating Point Comparison (The 'isclose' Trick)
# --------------------------------------------------
# NEVER check if floatA == floatB
a = 0.1 + 0.2
b = 0.3
print(f"\nIs 0.1 + 0.2 == 0.3? {a == b}") # False!
print(f"Using math.isclose: {math.isclose(a, b)}") # True
```

Note

Special Value: NaN (Not a Number)
In engineering calculations and data analysis, you will frequently encounter the value `NaN`. It stands for **"Not a Number"**.
What is it? It is a special floating-point value defined by the IEEE 754 standard to represent undefined or unrepresentable values.
When do we see it?

1. **High-School Math Errors**:
   - $0/0$ (Indeterminate form).
   - $\infty - \infty$.
   - $\sqrt{-1}$ (Square root of a negative number in real-number domain).
2. **Missing Data (Most Common)**: In Civil Engineering, sensor data often has gaps. If a sensor fails to record a temperature at 12:00 PM, libraries like Pandas will fill that cell with

`NaN` to indicate "Missing Value".

Dangerous Properties:

- **Propagation**: Any math operation with NaN results in NaN.

$$5 + \text{NaN} = \text{NaN}$$

  This is dangerous because one missing value can corrupt your entire calculation.

- **Inequality**: NaN is **not equal** to anything, including itself.

$$\text{NaN} == \text{NaN} \rightarrow \textbf{False}$$

  You cannot check for it using `if x == NaN`. You **must** use special functions: `math.isnan(x)` or `np.isnan(x)`.

9.2 Introduction to Arrays (NumPy)

We learned about `Lists` last week. Lists are great for general data, but they are **slow** and **clumsy** for math. If you have two lists of forces `F1` and `F2`, you cannot just do $F_{total} = F1 + F2$. Python will just join the lists together, not add the numbers.

For Engineering, we use **NumPy Arrays**.

9.2.1 Why Arrays?

- **Vectorization**: You can perform math on the whole array at once (e.g., `stress = force / area`).
- **Speed**: Arrays are up to 50x faster than lists.
- **Memory**: They use less RAM.

9.2.2 Common NumPy Functions

- `np.array([...])`: Converts a list to an array.
- `np.linspace(start, stop, num)`: Generates 'num' points evenly spaced between start and stop. This is **essential** for plotting graphs (e.g., generating x-coordinates for a beam).
- `np.zeros((rows, cols))`: Creates a matrix full of zeros. useful for initializing stiffness matrices.

Example (NumPy Basics).

Demonstrating the difference between Lists and Arrays, and generating data with `linspace`.

```
import numpy as np

# 1. Why NumPy? Lists vs Arrays
# ------------------------------
# Calculating stress = Force / Area for multiple beams
forces = [1000, 2000, 1500] # List
area = 0.5                  # Scalar
```

```python
# List approach (Error prone or needs loop)
# stresses = forces / area  <-- TypeError: unsupported operand type(s) for /: '
    list' and 'float'

# NumPy approach (Vectorization)
forces_arr = np.array([1000, 2000, 1500])
stresses = forces_arr / area # Element-wise operation!
print("Stresses (Array):", stresses)

# 2. Creating Arrays
# ------------------
# Manual
arr_1d = np.array([1, 2, 3])
arr_2d = np.array([[1, 2, 3], [4, 5, 6]]) # Matrix (2 rows, 3 cols)

print(f"\n1D Array shape: {arr_1d.shape}")
print(f"2D Array shape: {arr_2d.shape}")

# Generators (Very useful for graphs)
# linspace(start, stop, count) -> Generates 'count' numbers evenly spaced
x_values = np.linspace(0, 10, 5) # 0, 2.5, 5.0, 7.5, 10.0
print(f"\nLinspace (0 to 10): {x_values}")

# Zeros and Ones (Pre-allocating memory)
zeros_mat = np.zeros((3, 3)) # 3x3 matrix of zeros
print(f"\nZeros 3x3:\n{zeros_mat}")

# 3. Operations & Statistical Methods
# -----------------------------------
data = np.array([10.5, 12.0, 9.8, 11.2])
print(f"\nData: {data}")
print(f"Mean: {data.mean()}")
print(f"Standard Deviation: {data.std()}")
print(f"Max Index (argmax): {data.argmax()}") # Index of the max value
```

9.3 Introduction to DataFrames (Pandas)

While NumPy is for **Numbers**, Pandas is for **Data**. Think of a Pandas **DataFrame** as a programmable Excel sheet.

- It has **Rows** and **Columns**.
- Columns have names (Headers).
- Methods to load data: `pd.read_csv("data.csv")`, `pd.read_excel("data.xlsx")`.

9.3.1 Power of Pandas

- **Filtering**: `df[df["Stress"] > 200]` instantly gives you all failing rows.
- **Statistics**: `df.describe()` gives count, mean, std, min, max instantly.
- **Access**: `df.iloc[0]` gives the first row. `df["Name"]` gives the Name column.

Example (Pandas Intro).

Creating a simple Material Database and filtering it based on density.

```
import pandas as pd

# 1. Creating a DataFrame (Table)
# ------------------------------------
# We can create it from a Dictionary
data = {
    "Material": ["Concrete", "Steel", "Wood"],
    "Density_kg_m3": [2400, 7850, 600],
    "Elastic_Modulus_GPa": [30, 200, 11]
}

df = pd.DataFrame(data)
print("--- Material Table ---")
print(df)

# 2. Accessing Data
# --------------------
print("\n--- Column Access ---")
print(df["Density_kg_m3"])

print("\n--- Row Access (iloc) ---")
print("First Row (Concrete):")
print(df.iloc[0])

# 3. Simple Stats
# ----------------
print("\n--- Statistics ---")
print(df.describe())

# 4. Filtering (Conditionals)
# ------------------------------
# Find materials with Density > 1000
heavy_materials = df[df["Density_kg_m3"] > 1000]
print("\n--- Materials Heavy > 1000 kg/m3 ---")
print(heavy_materials)
```

10 Strings, Files, and Regular Expressions

Engineering is not just about numbers. It is also about **Text**.

- Structural Analysis software (SAP2000, ETABS) exports results in huge text files (.out, .log).
- Sensors visualize data in CSV (Comma Separated Values) format.
- Reports need to be generated automatically.

In Week 8, we learn how to deal with the "Text" part of programming.

10.1 String Manipulation

A string is just a sequence of characters. In Python, there

10.1.1 Key Methods

- **Slice**: `s[0:4]` works exactly like lists. You can grab specific parts of a text.
- **Strip**: `s.strip()` removes invisible "garbage" characters from the beginning and end of a string (like spaces and newlines
`n`). This is crucial when reading files.
- **Split**: `s.split(",")` chops a string into a list of smaller strings based on a delimiter. Perfect for CSV files.
- **Replace**: `s.replace("m", "")` replaces specific characters. Useful for removing units from numbers.

10.1.2 Formatting Reports (f-strings)

Instead of manually adding strings (`"Load: " + str(load) + " kN"`), which is ugly and slow, use **f-strings**. They allow you to control precision directly.

- `f"{value:.2f}"` → Rounds to 2 decimal places.
- `f"{value:03d}"` → Pads with zeros (e.g., 005).

Example (String Operations).

Cleaning, splitting, and formatting engineering data strings.

```
# String Manipulation for Engineers
# ---------------------------------
# In engineering, we often get data as messy text strings.
# We need to clean, split, and format them.

# 1. The "Slice" (Accessing parts of a string)
# --------------------------------------------
sample_code = "BEAM-2024-SECTION-A"
print(f"Original: {sample_code}")
print(f"First 4 chars: {sample_code[0:4]}")  # "BEAM"
print(f"Last char: {sample_code[-1]}")       # "A"

# 2. Cleaning Data (.strip)
# -------------------------
# Essential when reading files! '  500  \n' -> '500'
```

```
messy_input = "    500.5    \n"
clean_input = messy_input.strip()
print(f"\nMessy: '{messy_input}'")
print(f"Clean: '{clean_input}'")

# 3. Splitting Data (.split)
# ------------------------------
# Essential for CSV files or log strings
data_line = "Concrete,C30,2400,0.2"
values = data_line.split(",") # Splits where it finds ","
print(f"\nSplit Data: {values}")
# Note: They are still Strings! We need to cast them.
density = float(values[2])
print(f"Density (float): {density}")

# 4. Replacement (.replace)
# ------------------------------
# Fixing units or typos
report = "Length is 5m, Width is 2m"
numeric_report = report.replace("m", "") # Remove 'm'
print(f"\nReport: {report}")
print(f"Numeric ready: {numeric_report}")

# 5. F-String Formatting (Reporting)
# ------------------------------
load = 123.456789
d = 2.0
# We want: "Load: 123.46 kN | Depth: 2 m"
# :.2f means "Float with 2 decimal places"
formatted = f"Load: {load:.2f} kN | Depth: {d:.0f} m"
print(f"\nFormatted Report: {formatted}")
```

10.2 File Input/Output (I/O)

Variables live in RAM. When you turn off the computer, they vanish. To save them, we write them to Files (Hard Disk).

10.2.1 The `with open(...)` Pattern

In older languages, you had to manually open and close files. If your code crashed before closing, the file could get corrupted. Python uses the `with` statement, which acts as a safety wrapper. It closes the file automatically, even if errors occur.

Modes:

- `"r"` (Read): Errors if file missing. Default mode.
- `"w"` (Write): **Danger!** Creates a new file. If file exists, **deletes everything** in it.
- `"a"` (Append): Adds to the end of an existing file.

Example (Reading and Writing Files).

Writing a log file and reading it back to detect high stress events.

```
# File I/O (Input/Output) Basics
# ------------------------------
# How to read and write text files safely.
```

```
filename = "site_log.txt"

# 1. Writing to a File ('w' mode)
# ------------------------------------
# 'w' mode deletes the file content if it exists and starts fresh!
# The 'with' statement automatically closes the file (SAFE).
print("Writing data to file...")
with open(filename, "w") as f:
    f.write("Time,Temperature,Stress\n") # Header
    f.write("12:00,25.5,100\n")          # Data
    f.write("12:15,26.0,110\n")
    f.write("12:30,26.2,115\n")
print("Write complete.\n")

# 2. Appending to a File ('a' mode)
# ------------------------------------
# 'a' adds to the end without deleting existing content.
with open(filename, "a") as f:
    f.write("12:45,25.8,105\n")

# 3. Reading a File ('r' mode)
# ------------------------------------
print("Reading file line by line:")
with open(filename, "r") as f:
    # f.readlines() reads all lines into a LIST
    all_lines = f.readlines()

for line in all_lines:
    # line looks like "12:00,25.5,100\n"
    # 1. Strip the invisible newline '\n' at the end
    clean_line = line.strip()

    # 2. Split by comma
    parts = clean_line.split(",")

    print(f"Row: {parts}")

    # Example: Check for high stress (ignoring Header)
    if parts[0] != "Time":
        stress = float(parts[2])
        if stress > 110:
            print(f"  -> WARNING: High Stress detected at {parts[0]}!")

# 4. Deleting the file (Cleanup)
# ------------------------------------
import os
# os.remove(filename) # Uncomment to delete the file after running
```

10.3 Regular Expressions (Regex)

Sometimes, `.split()` is not enough. Imagine searching for a pile diameter in a PDF report. It could be written as:

- "D=80cm"
- "Dia: 800 mm"
- "Diameter 0.8m"

Standard string methods fail here because the format varies. **Regex** allows you to define a **Pattern** instead of a specific text.

Think of it as "Ctrl+F" on steroids.

10.3.1 Common Patterns

We use the `re` library.

- `d` : Matches any digit (0-9).
- `w` : Matches any letter or number.
- `.` : Matches **anything**.
- `+` : Matches the previous pattern **one or more times**. (e.g., `d+` matches "1", "12", "100").

Example (Regex for Extracting Data).

extracting error codes and geometrical dimensions from mixed text using `re.search` and `re.findall`.

```
import re # Import the Regex module

# Regular Expressions (Regex)
# ---------------------------
# The "Smart Find" tool.
# Used when .split() is too complicated because the text is inconsistent.

# Scenario: Extracting error codes from messy logs
log_entry = "Error happened at [Disk-1] with Code: #404 (Timeout)"
print(f"Log: {log_entry}")

# Goal: Find the number "404"
# Naive way (splitting by space) would fail because of "#" or "("

# 1. re.search (Find the first match)
# -----------------------------------
# Pattern: #\d+
# #   -> Look for a hashtag
# \d  -> Look for a digit (0-9)
# +   -> Look for ONE or MORE digits
match = re.search(r"#\d+", log_entry)

if match:
    print(f"Found code string: {match.group()}") # Returns "#404"
    # To get just the number, we can slice:
    print(f"Code number: {match.group()[1:]}")   # Returns "404"
else:
    print("No code found.")


# 2. re.findall (Find ALL matches)
# --------------------------------
# Scenario: Extracting beam dimensions from a text
# "Beam A is 300x500 mm and Beam B is 400x600 mm"
text = "Section 1: 300x500mm, Section 2: 25.5x40.0cm"

# Pattern Explanation:
# \d+     -> Digits
# \.?     -> Optional dot (for decimals like 25.5)
# \d*     -> Optional decimals after dot
```

```
# x       -> The letter 'x'
pattern = r"\d+\.?\d*x\d+\.?\d*"

dimensions = re.findall(pattern, text)
print(f"\nText: {text}")
print(f"Found Dimensions: {dimensions}")

# 3. Simple Patterns Guide
# ------------------------
# \d   : Any digit (0-9)
# \w   : Any letter or number (A-Z, 0-9)
# .    : Any character (dot means 'anything')
# +    : One or more
# *    : Zero or more
```

11 Functions and Error Handling

In the previous weeks, we wrote code sequantially. Line 1, Line 2, Line 3... But as Engineering projects get complex, writing everything in one long script turns into a nightmare. Week 9 introduces the most important concept in efficient programming: **Functions**.

11.1 What is a Function?

A function is a reusable block of code that performs a specific task. You have already used them: `print()`, `len()`, `math.sqrt()`. Now, you will build your own.

11.1.1 The Factory Analogy

Imagine a **Function** as a **Machine** in a factory (e.g., a Concrete Mixer).

- **Input (Arguments)**: It takes raw materials (Water, Cement, Sand).
- **Process (Body)**: Inside the machine, it mixes them. You don't need to know the internal mechanics, you just trust it works.
- **Output (Return Value)**: It ejects the product (Fresh Concrete).

11.1.2 Why use functions?

DRY Principle: Don't Repeat Yourself. If you need to calculate the Moment of Inertia of a T-beam in 10 different places in your code, do not copy-paste the formula 10 times. Write a function `calc_inertia_T()` and call it 10 times. If you find a bug in the formula, you fix it in **one place**, and it updates everywhere.

11.2 Anatomy of a Function

To define a function in Python, we use the `def` keyword.

```
def function_name(parameter1, parameter2):
    # Body of the function
    result = parameter1 + parameter2
    return result
```

11.2.1 Return vs Print

This is the #1 confusion for beginners.

- **Print**: Shows the result on the screen for the **Human**. The computer forgets it immediately.
- **Return**: Gives the result back to the **Computer**. The program can save it in a variable and use it for the next calculation.

Analogy: `Print` is like a chef shouting "Soup is ready!". `Return` is the waiter actually bringing the soup to your table.

Example (Basic Function Structure).

Inputs, Outputs, Docstrings (Documentation), and Default Arguments.

```
# Anatomy of a Function
```

```
# ------------------------
# Analogy: A Function is like a "Concrete Mixer Truck".
# Input (Arguments): Water, Cement, Aggregate.
# Process: Mixing/Rotating.
# Output (Return): Fresh Concrete.

def calculate_beam_weight(length, width, height, density=2400):
    """
    Calculates the weight of a beam.

    Parameters:
    length (float): Length in meters.
    width (float): Width in meters.
    height (float): Height in meters.
    density (float): Density in kg/m3 (Default is Concrete: 2400).

    Returns:
    float: The total weight in kg.
    """

    # 1. Processing (The Logic)
    volume = length * width * height
    weight = volume * density

    # 2. Output (Return)
    return weight

# Using the function (Calling the machine)
# ----------------------------------------

# Scenario 1: Standard Concrete Beam using Positional Arguments
wt_1 = calculate_beam_weight(5.0, 0.3, 0.5)
print(f"Beam 1 Weight (Concrete): {wt_1} kg")

# Scenario 2: Steel Beam (Overriding the default density) using Keyword Arguments
# Keyword arguments make it readable and order doesn't matter much.
wt_2 = calculate_beam_weight(length=4.0, width=0.1, height=0.2, density=7850)
print(f"Beam 2 Weight (Steel): {wt_2} kg")

# Scenario 3: What if we didn't use 'return'?
def useless_calculator(a, b):
    result = a + b
    print(f"Result is {result}")
    # No return statement implies "return None"

value = useless_calculator(10, 20)
print(f"Captured Value: {value}") # Output: None. We cannot use this result in
    other calculations!
```

11.3 Variable Scope (Local vs Global)

Where a variable "lives" matters.

- **Global Scope** (The Construction Site): Variables defined in the main body. Everyone can see them.
- **Local Scope** (The Site Office): Variables defined **inside** a function. They only exist while that function is running. Once the function finishes, they are destroyed.

Example (Scope and Shadowing).

Demonstrating why a function cannot access local variables of another function, and how local variables can "shadow" global ones.

```
# Variable Scope (Local vs Global)
# ---------------------------------
# Analogy:
# Global Variable: The "Site Crane". Everyone on the site can see and use it.
# Local Variable: The "Site Engineer's Personal Pen". Only the engineer in that
    office can use it.

# 1. Global Scope
project_name = "Suspension Bridge" # This is Global

def print_project_info():
    # We can READ global variables inside a function
    print(f"Working on: {project_name}")

print_project_info()


# 2. Local Scope
def calculate_stress(force, area):
    # 'safety_factor' is created INSIDE the function.
    # It is LOCAL. It dies when the function finishes.
    safety_factor = 1.5
    design_stress = (force / area) * safety_factor
    return design_stress

stress = calculate_stress(1000, 10)
print(f"Calculated Stress: {stress}")

# Error Scenario
# print(safety_factor)
# NameError: name 'safety_factor' is not defined.
# The main program cannot reach inside the function to get the pen!


# 3. Shadowing (Tricky!)
# ----------------------
limit = 100 # Global

def check_limit():
    limit = 50 # Local variable with SAME name (Shadows the global one)
    print(f"Inside function, limit is: {limit}")

check_limit()
print(f"Outside function, limit is: {limit}") # Still 100! The global one was
    untouched.
```

11.4 Advanced Function Arguments

Sometimes, we need more flexibility in how we call our machines.

11.4.1 Default Arguments

Some parameters can have a "fallback" value. If the user doesn't specify them, the factory uses the default.

```
def mix_concrete(cement, water, admixture="None"):
    # If user forgets admixture, we assume "None"
    ...
```

11.4.2 Flexible Arguments (*args)

What if we don't know how many inputs we will get? Imagine a function that calculates the **Total Weight** of items in a truck. The truck might have 1 item, or 50 items. We cannot define `def weight(item1, item2, item3...)`.

We use `*args`. The asterisk `*` tells Python: "Acccept any number of positional arguments and pack them into a Tuple named `args`".

Analogy: A "Universal Socket Wrench". It adapts to fit however many bolts you throw at it.

Example (Integral Calculator with *args).

A smart function that adapts its formula (Linear, Quadratic, Cubic) based on the number of coefficients provided by the user using `*args`.

```
def calculate_integral(x1, x2, *coeffs):
    """
    Calculates the definite integral of a polynomial function between x1 and x2.

    The function form depends on the number of coefficients provided:
    1 arg (a)          -> f(x) = ax
    2 args (a, b)      -> f(x) = ax + b
    3 args (a, b, c)   -> f(x) = ax^2 + bx + c
    4 args (a, b, c, d)-> f(x) = ax^3 + bx^2 + cx + d
    5 args (a..e)      -> f(x) = ax^4 + bx^3 + cx^2 + dx + e

    Parameters:
    x1 (float): Lower limit of integration.
    x2 (float): Upper limit of integration.
    *coeffs: Coefficients of the polynomial (a, b, c, ...).

    Returns:
    float: The value of the definite integral.
    None: If the number of coefficients is not supported.
    """

    # Helper function to calculate F(x) (Antiderivative value at x)
    def antiderivative(x, c):
        n = len(c)
        val = 0

        # Case 1: f(x) = ax (Special case per instructions)
        if n == 1:
            a = c[0]
            # Integral of ax is (a/2)x^2
            val = (a / 2) * x**2

        # Case 2: Linear f(x) = ax + b
        elif n == 2:
            a, b = c
            val = (a / 2) * x**2 + b * x

        # Case 3: Quadratic f(x) = ax^2 + bx + c
        elif n == 3:
```

```
            a, b, c_const = c
            # Integral: (a/3)x^3 + (b/2)x^2 + cx
            val = (a / 3) * x**3 + (b / 2) * x**2 + c_const * x

        # Case 4: Cubic f(x) = ax^3 + bx^2 + cx + d
        elif n == 4:
            a, b, c_const, d = c
            val = (a / 4) * x**4 + (b / 3) * x**3 + (c_const / 2) * x**2 + d * x

        # Case 5: Quartic f(x) = ax^4 + ... + e
        elif n == 5:
            a, b, c_const, d, e = c
            val = (a / 5) * x**5 + (b / 4) * x**4 + (c_const / 3) * x**3 + (d / 2)
                 * x**2 + e * x

        else:
            raise ValueError(f"Unsupported number of coefficients: {n}. Please
                provide 1 to 5 coefficients.")

        return val

    try:
        # Calculate F(x2) - F(x1)
        # We pass 'coeffs' tuple to our inner helper function
        result = antiderivative(x2, coeffs) - antiderivative(x1, coeffs)
        return result

    except ValueError as e:
        print(f"Error: {e}")
        return None
    except TypeError:
        print("Error: Inputs must be numbers.")
        return None

# Main Execution Block
# ---------------------
if __name__ == "__main__":
    print("--- Integral Calculator ---")

    # 1. Linear Special: f(x) = 5x, from 0 to 10
    res1 = calculate_integral(0, 10, 5)
    print(f"Integral of 5x (0 to 10): {res1}")

    # 2. Linear Standard: f(x) = 2x + 3, from 0 to 5
    # Integral = x^2 + 3x = (25+15) - 0 = 40
    res_lin = calculate_integral(0, 5, 2, 3)
    print(f"Integral of 2x + 3 (0 to 5): {res_lin}")

    # 3. Quadratic: f(x) = 1x^2 + 0x + 0, from 0 to 3
    res2 = calculate_integral(0, 3, 1, 0, 0)
    print(f"Integral of x^2 (0 to 3): {res2}")

    # 4. Error Case (Unsupported length, e.g., 6 coefficients)
    print("\n--- Testing Error Handling ---")
    calculate_integral(0, 5, 1, 2, 3, 4, 5, 6) # Should print error
```

11.5 Error Handling (Try / Except)

In Civil Engineering, we design systems not just for normal loads, but for failures. We assume earthquakes *will* happen. Similarly, in Programming, we must assume **Errors will happen**. Users will enter text instead of numbers, files will be missing, or internet connections will drop.

Without error handling, a Python program "crashes" (stops immediately and displays red error

text) when it hits an error. This is unacceptable for professional software.

11.5.1 The Analogy: Runaway Truck Ramp

Imagine your code is a **Heavy Truck** driving down a steep highway.

- **The Main Highway (`try`)**: This is the normal path. The truck attempts to drive here. As long as the brakes work, it stays on this road.
- **Brake Failure (The Exception)**: Suddenly, a critical problem occurs (e.g., Division by Zero, File Not Found). The truck cannot continue on the highway safely.
- **The Runaway Ramp (`except`)**: This is a dedicated gravel path designed to catch the truck if things go wrong. Instead of crashing into other cars (crashing the Operating System or losing data), the truck steers into the ramp and stops safely. The trip is interrupted, but **no catastrophe occurs**.

11.5.2 The `finally` Block

Sometimes, we need to clean up regardless of what happened. *Analogy*: Whether the truck finished the trip or stopped in the ramp, the **Logbook** must be signed. The `finally` block runs **always**, ensuring files are closed or connections are disconnected.

Example (Catching Errors).

Handling `ZeroDivisionError` and `TypeError` without crashing the program, using the Try-Except safety mechanism.

```
# Error Handling (Try-Except)
# ---------------------------
# Analogy: "The Safety Net".
# If a trapeze artist falls (Error occurs), the net catches them (Except block),
# so the show doesn't stop (Program doesn't crash).

def safe_division(load, area):
    try:
        stress = load / area
        return stress
    except ZeroDivisionError:
        print("CRITICAL ERROR: Area cannot be zero! Infinite stress!")
        return None
    except TypeError:
        print("ERROR: Please input numbers, not text!")
        return None

# Test 1: Normal
print(f"Test 1: {safe_division(100, 10)}")

# Test 2: Zero Area (Would normally crash the program)
print(f"Test 2: {safe_division(100, 0)}")

# Test 3: Wrong Input
print(f"Test 3: {safe_division(100, 'ten')}")

print("\nProgram continued running successfully after errors!")
```

12 File Handling and CSVs

Data comes in many shapes. As engineers, we rarely type data into the code manually. instead, we **Read** it from files. Week 10 covers the standard methods to handle external data files, with a heavy focus on **CSV** (Comma Separated Values).

12.1 Basic File Operations

We discussed `open()` briefly in Week 8. Now we will use it in more detail.

- **Text Files (.txt)**: Good for simple notes or unstructured logs (mostly logs. Developers always use plain text formats. However in practice, the extension of the file is defines the purpose of the file. Of course there could be exceptions but .out, .log, .txt files can be opened by any text editor.).
- **Binary Files (.bin, .jpg)**: Images or compiled programs. We won't touch these yet.
- **Structured Text (.csv, .json, .xml)**: The standard for data exchange (the most easy and proper whay to handle data or datasets).

12.2 The CSV Format

CSV is the universal language of Engineering Data. It is just a text file where columns are separated by commas.

```
Time, Force, Displacement
10.0, 500, 2.5
10.1, 510, 2.6
```

Note

In CSV files, columns are usually separated by commas. However, this is not a strict rule. You can easily see that semicolons are also used as separators. To open a CSV file with semicolons or any other separator, you can use the following parameters:

```
with open('data.csv', 'r', newline='') as file:
    reader = csv.reader(file, delimiter=';')
```

12.2.1 Why not just use Excel (.xlsx)?

Excel files are complex, binary "zip" archives full of formatting, fonts, and heavy metadata. CSV is **plain text**. It is lightweight, fast, and can be opened by **any** software (Python, MATLAB, AutoCAD, Excel, Notepad). The adtavantages of Excel is that graphing and formatting is easier. However, when you get familiar with Python, you will find it much easier to handle data with Python. In addition to that, you'll be able to generate graphs that all in a standard format which its so important for data presentation.

12.3 Reading CSV Files

Python provides a built-in library called `csv`.

12.3.1 1. The `csv.reader`

Reads the file row by row. Each row is returned as a **List of Strings**.

1. You must open the file first.
2. You pass the file object to `csv.reader()`.
3. You iterate through it with a `for` loop.

Example (CSV Basics).

Demonstrating Manual Parsing vs. the `csv` Library.

```
import csv

# WEEK 10: CSV File Operations
# ------------------------------
# "CSV" stands for Comma Separated Values.
# It is the most common format for Engineering Data exchange.

filename = "code/week10_concrete_data.csv"

# 1. Reading CSV Manually (The Hard Way)
# ----------------------------------------
# Useful to understand what CSV actually is: Just text with commas!
print("--- Method 1: Manual Parsing ---")
with open(filename, "r") as f:
    # Read first 3 lines only
    for i in range(3):
        line = f.readline().strip()
        columns = line.split(",")
        print(f"Row {i}: {columns}")

# 2. Reading with 'csv' Library (The Standard Way)
# --------------------------------------------------
# Handles edge cases (like commas inside text) automatically.
print("\n--- Method 2: csv.reader ---")
with open(filename, "r") as f:
    reader = csv.reader(f)

    # Getting the Header
    header = next(reader) # Skips the first line and saves it
    print(f"Header: {header}")

    # Read data
    row_count = 0
    for row in reader:
        row_count += 1
        # Let's print only the first 2 data rows
        if row_count <= 2:
            print(f"Data {row_count}: {row}")

# 3. Writing to CSV
# -------------------
output_file = "code/week10_report.csv"
data_to_write = [
    ["Test_ID", "Result", "Notes"],
    [1, "Pass", "Good surface"],
    [2, "Fail", "Cracked early"]
]
```

```
print(f"\n--- Writing to {output_file} ---")
with open(output_file, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data_to_write)
print("Write complete.")
```

12.3.2 2. The `csv.DictReader` (Pro Move)

Instead of remembering that "Strength" is Column 8 (e.g., `row[8]`), `DictReader` lets you use column names (e.g., `row["Strength"]`). This makes your code **readable** and **resistant to column changes**.

12.4 Application: Statistical Analysis

Let's analyze a real dataset of 300 Concrete Cylinder tests. **Scenario**: You are the Quality Control Engineer. You have a messy raw data file. You need to:

- Filter out the tests that are NOT 28-day cure time.
- Calculate the Mean and Standard Deviation of the strength.
- Decide if the batch Passes or Fails.

Example (Statistics on CSV Data).

A complete script to read, filter, parse, and statistically analyze engineering data.

```
import csv
import statistics

# WEEK 10: Parsing & Statistics Application
# ------------------------------------------
# Goal: Read 300 concrete tests, filter by 28-day strength,
# and calculate statistics.

filename = "code/week10_concrete_data.csv"

# Lists to hold our data columns
strengths_28_days = []
batch_codes = []

print(f"Reading {filename}...")

with open(filename, "r") as f:
    reader = csv.DictReader(f) # DictReader map header to values!
    # row is now like: {'Sample_ID': 'CYL-1001', 'Compressive_Strength_MPa':
        '35.2', ...}

    for row in reader:
        try:
            # Parse Data
            days = int(row["Cure_Time_Days"])
            strength = float(row["Compressive_Strength_MPa"])
            batch = row["Batch_Code"]

            # Application Logic: We only care about standard 28-day tests
            if days == 28:
                strengths_28_days.append(strength)
```

```
                batch_codes.append(batch)

        except ValueError:
            print("Skipping invalid row...")

# Statistical Analysis
# ---------------------
count = len(strengths_28_days)
avg = statistics.mean(strengths_28_days)
std_dev = statistics.stdev(strengths_28_days)
min_val = min(strengths_28_days)
max_val = max(strengths_28_days)

# Reporting
print("\n--- 28-Day Concrete Strength Analysis ---")
print(f"Total Samples Analyzed: {count}")
print(f"Mean Strength:      {avg:.2f} MPa")
print(f"Standard Dev:       {std_dev:.2f} MPa")
print(f"Range:              {min_val} - {max_val} MPa")

# Acceptance Check (Example C30/37 Concrete)
# A simplified rule: Mean must be >= 38 MPa (fck + margin)
target = 38.0
if avg >= target:
    print(f"\nRESULT: BATCH PASSED (Mean > {target})")
else:
    print(f"\nRESULT: BATCH FAILED (Mean < {target})")
```

13 Data Visualization

"A picture is worth a thousand numbers." In Engineering, we deal with massive datasets. Looking at a spreadsheet with 10,000 rows tells you nothing. Plotting that data reveals:

- **Trends**: Is the bridge moving over time?
- **Outliers**: Is one sensor giving garbage data?
- **Correlations**: Does adding more cement actually increase strength?

In Week 11, we will learn how to turn data into insight using Python.

13.1 Libraries

- **Matplotlib** (`plt`): The grandfather of Python plotting. Powerful but requires a lot of code for complex plots.
- **Seaborn** (`sns`): Built on top of Matplotlib. Designed for **Statistical Data Visualization**. It makes beautiful plots in one line of code.

13.2 Distributions (Histograms)

The first thing you should do with any new dataset is to check the **Distribution**. Is your data `Normal` (Bell curve)? Or is it `skewed`?

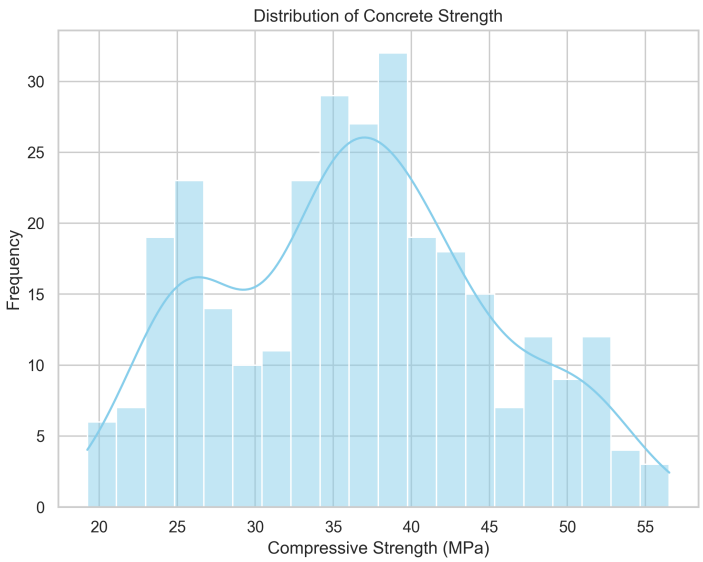


Figure 1: Distribution of Concrete Strength. Notice the Bell Curve shape.

13.3 Correlations (Scatter Plots)

To see if two variables are related, we use Scatter Plots. Below, we check if heavier samples carry more load. The color shows the Cure Time.

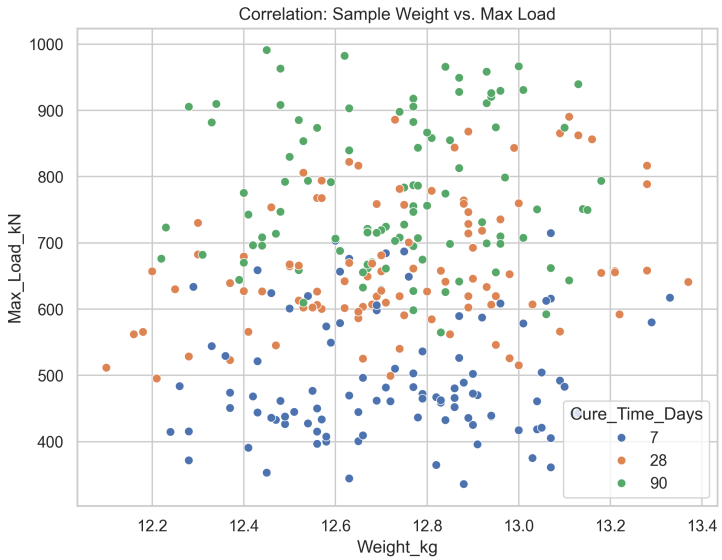


Figure 2: Scatter Plot: Weight vs Load. There is a clear positive correlation.

13.4 Comparing Groups (Box Plots)

How does Cement Type affect strength? A Box Plot shows the Median, Quartiles, and Outliers.

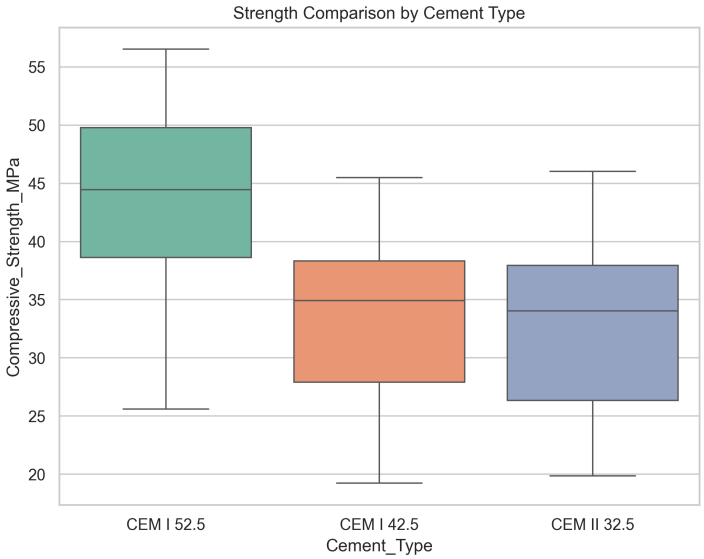


Figure 3: Box Plot: Strength by Cement Type. CEM I 52.5 is clearly stronger.

13.5 The Engineering Dashboard (Subplots)

Professional reports often combine multiple plots into one figure using `plt.subplots()`.

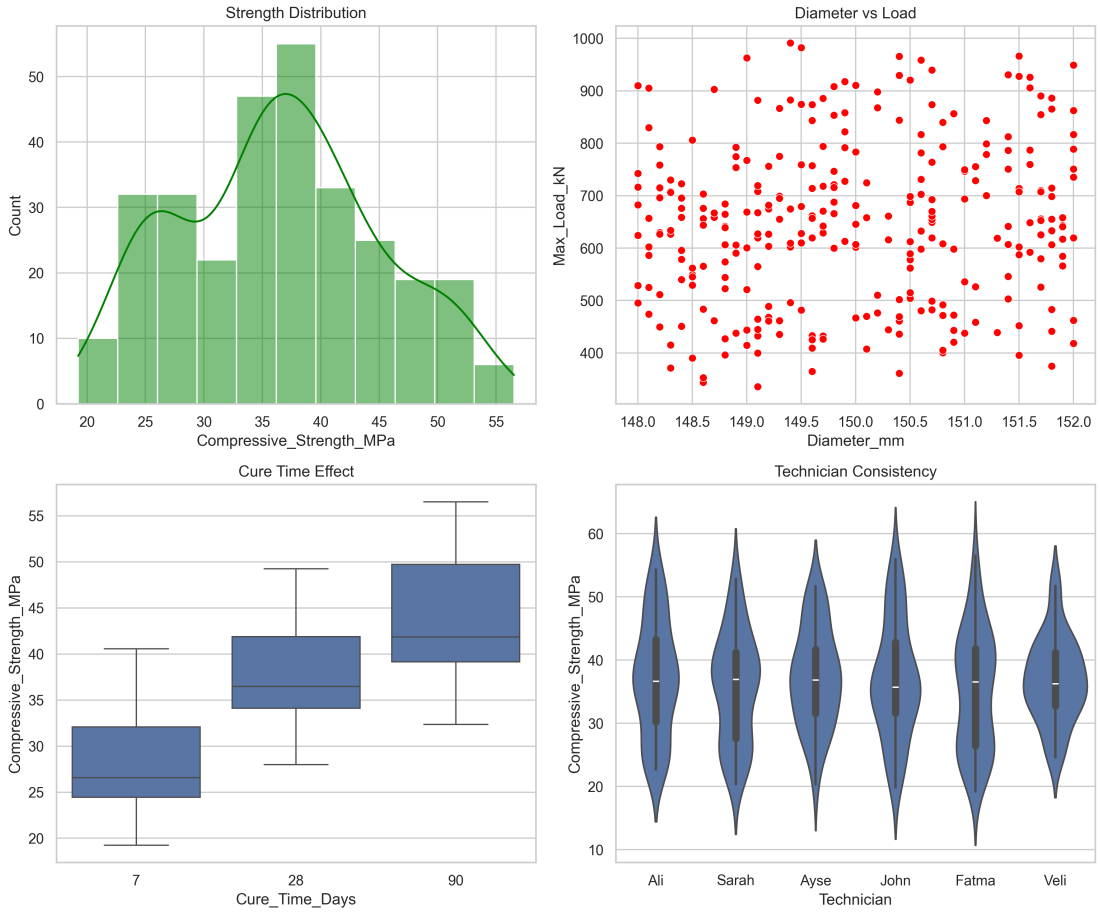


Figure 4: A complete Engineering Analysis Dashboard.

Example (Generating Plots).

The Python code used to generate all the figures above.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

# Ensure the output directory exists
output_dir = "../En-en/fig"
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# 1. Load Data
# ------------
df = pd.read_csv("week10_concrete_data.csv")

# Set a nice theme
sns.set_theme(style="whitegrid")
```

```
# 2. Histogram (Distribution of Strength)
# ---------------------------------------
plt.figure(figsize=(8, 6))
# Histogram with a Kernel Density Estimate (KDE) line
sns.histplot(data=df, x="Compressive_Strength_MPa", kde=True, bins=20, color="
    skyblue")
plt.title("Distribution of Concrete Strength")
plt.xlabel("Compressive Strength (MPa)")
plt.ylabel("Frequency")
plt.savefig(f"{output_dir}/week11_hist.png", dpi=300)
print("Saved: week11_hist.png")

# 3. Scatter Plot (Correlation: Weight vs Load)
# ---------------------------------------------
plt.figure(figsize=(8, 6))
# Does heavier concrete carry more load?
sns.scatterplot(data=df, x="Weight_kg", y="Max_Load_kN", hue="Cure_Time_Days",
    palette="deep")
plt.title("Correlation: Sample Weight vs. Max Load")
plt.savefig(f"{output_dir}/week11_scatter.png", dpi=300)
print("Saved: week11_scatter.png")

# 4. Box Plot (Comparison by Category)
# ------------------------------------
plt.figure(figsize=(8, 6))
# Compare strength across different cement types
sns.boxplot(data=df, x="Cement_Type", y="Compressive_Strength_MPa", palette="Set2"
    )
plt.title("Strength Comparison by Cement Type")
plt.savefig(f"{output_dir}/week11_boxplot.png", dpi=300)
print("Saved: week11_boxplot.png")

# 5. Bar Plot (Average Strength by Cure Time)
# -------------------------------------------
plt.figure(figsize=(8, 6))
sns.barplot(data=df, x="Cure_Time_Days", y="Compressive_Strength_MPa", errorbar="
    sd", capsize=.1)
plt.title("Strength Gain over Time (Mean + Std Dev)")
plt.savefig(f"{output_dir}/week11_barplot.png", dpi=300)
print("Saved: week11_barplot.png")

# 6. Subplots (Engineering Dashboard)
# -----------------------------------
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Plot A: Hist
sns.histplot(data=df, x="Compressive_Strength_MPa", kde=True, ax=axs[0, 0], color=
    "green")
axs[0, 0].set_title("Strength Distribution")

# Plot B: Scatter
sns.scatterplot(data=df, x="Diameter_mm", y="Max_Load_kN", ax=axs[0, 1], color="
    red")
axs[0, 1].set_title("Diameter vs Load")

# Plot C: Box
sns.boxplot(data=df, x="Cure_Time_Days", y="Compressive_Strength_MPa", ax=axs[1,
    0])
axs[1, 0].set_title("Cure Time Effect")

# Plot D: Violin (Advanced Boxplot)
sns.violinplot(data=df, x="Technician", y="Compressive_Strength_MPa", ax=axs[1,
    1])
```

```
axs[1, 1].set_title("Technician Consistency")

plt.tight_layout()
plt.savefig(f"{output_dir}/week11_dashboard.png", dpi=300)
print("Saved: week11_dashboard.png")
```

14 Practice

For those who are trying to learn any Programming Language, there are a lot of exercises and problems. In this context the Project Euler (https://projecteuler.net/archives) is a perfect source. In this lecture, each student will try to solve same problem. As you can guess, this and any other problem can be solved in different ways. The thing that will create the difference is the efficiency of the code. Each student who successfully solves the problem will be sorted by the CPU time. The following problem is a just an example. Each practice group will be assigned a different problem.

Question 3

Smallest Multiple 2520 is the smallest number that can be divided by each of the numbers from 1 to 10 without any remainder. What is the smallest positive number that is evenly divisible by all of the numbers from 1 to 20?

15 Structural Analysis with Python

In this lecture, the demonstration of the structural analysis with Python will be done. The OpenSeesPy is a Python interface to the OpenSees (Open System for Earthquake Engineering Simulation) finite element analysis software. Contrary to most of the commercial software, OpenSeesPy is an open-source and reliable software that is widely used in both academia and industry.

As today, OpenSeesPy works best with Python 3.12. However, the recent version of the Python is 3.13. So first of all, we need to get the Python 3.12. The first option is to setting previous version as default version of Python. The second option is to use virtual environments. In this example, we will use the second option to prevent any conflict with the other Python packages.

```
# Install Python 3.12
winget install Python.Python.3.12
# Check the Python installation
py -3.12 --version
# Create a virtual environment
py -3.12 -m venv myenv312
# Activate the virtual environment
myenv312\Scripts\activate
# Upgrade pip
python -m pip install --upgrade pip
# Install the required packages
pip install openseespy==3.7.1.2 opsvis numpy pandas matplotlib requests scikit-learn
```

After the installation, we can start to use OpenSeesPy. The following code is a simple example of how to use OpenSeesPy to analyze a simple two storey, three bay 2D frame.

```
import openseespy.opensees as ops
import opsvis as opsv

import matplotlib.pyplot as plt

ops.wipe()
ops.model('basic', '-ndm', 2, '-ndf', 3)

colL, girL = 4., 6.

Acol, Agir = 2.e-3, 6.e-3
IzCol, IzGir = 1.6e-5, 5.4e-5

E = 200.e9

Ep = {1: [E, Acol, IzCol],
      2: [E, Acol, IzCol],
      3: [E, Agir, IzGir]}

# Nodes
# Ground floor nodes (y=0)
ops.node(1, 0., 0.)           # Left bottom corner
ops.node(2, girL, 0.)         # Middle bottom
ops.node(3, 2*girL, 0.)       # Middle bottom 2
ops.node(4, 3*girL, 0.)       # Right bottom corner

# 1. floor nodes (y=colL)
ops.node(5, 0., colL)         # Left 1. floor
ops.node(6, girL, colL)       # Middle 1. floor
ops.node(7, 2*girL, colL)     # Middle 1. floor 2
ops.node(8, 3*girL, colL)     # Right 1. floor

# 2. floor nodes (y=2*colL)
ops.node(9, 0., 2*colL)       # Left 2. floor
ops.node(10, girL, 2*colL)    # Middle 2. floor
ops.node(11, 2*girL, 2*colL)  # Middle 2. floor 2
```

```
ops.node(12, 3*girL, 2*colL)  # Right 2. floor

# Ground floor nodes fixed
ops.fix(1, 1, 1, 1)
ops.fix(2, 1, 1, 0)
ops.fix(3, 1, 1, 0)
ops.fix(4, 1, 1, 0)

opsv.plot_model()
plt.title('plot_model before defining elements')
plt.savefig('model_before_elements.png', dpi=300, bbox_inches='tight')
plt.clf()

ops.geomTransf('Linear', 1)

# Columns - Ground floor to 1. floor
ops.element('elasticBeamColumn', 1, 1, 5, Acol, E, IzCol, 1)   # Left column
ops.element('elasticBeamColumn', 2, 2, 6, Acol, E, IzCol, 1)   # Middle column 1
ops.element('elasticBeamColumn', 3, 3, 7, Acol, E, IzCol, 1)   # Middle column 2
ops.element('elasticBeamColumn', 4, 4, 8, Acol, E, IzCol, 1)   # Right column

# Columns - 1. floor to 2. floor
ops.element('elasticBeamColumn', 5, 5, 9, Acol, E, IzCol, 1)   # Left column
ops.element('elasticBeamColumn', 6, 6, 10, Acol, E, IzCol, 1)  # Middle column 1
ops.element('elasticBeamColumn', 7, 7, 11, Acol, E, IzCol, 1)  # Middle column 2
ops.element('elasticBeamColumn', 8, 8, 12, Acol, E, IzCol, 1)  # Right column

# Beams - 1. floor
ops.element('elasticBeamColumn', 9, 5, 6, Agir, E, IzGir, 1)   # 1. bay
ops.element('elasticBeamColumn', 10, 6, 7, Agir, E, IzGir, 1)  # 2. bay
ops.element('elasticBeamColumn', 11, 7, 8, Agir, E, IzGir, 1)  # 3. bay

# Beams - 2. floor
ops.element('elasticBeamColumn', 12, 9, 10, Agir, E, IzGir, 1)  # 1. bay
ops.element('elasticBeamColumn', 13, 10, 11, Agir, E, IzGir, 1) # 2. bay
ops.element('elasticBeamColumn', 14, 11, 12, Agir, E, IzGir, 1) # 3. bay

Px = 2.e+3
Wy = -10.e+3
Wx = 0.

# Apply distributed loads to all beams
Ew = {9: ['-beamUniform', Wy, Wx],    # 1. floor 1. bay
      10: ['-beamUniform', Wy, Wx],   # 1. floor 2. bay
      11: ['-beamUniform', Wy, Wx],   # 1. floor 3. bay
      12: ['-beamUniform', Wy, Wx],   # 2. floor 1. bay
      13: ['-beamUniform', Wy, Wx],   # 2. floor 2. bay
      14: ['-beamUniform', Wy, Wx]}   # 2. floor 3. bay

ops.timeSeries('Constant', 1)
ops.pattern('Plain', 1, 1)

# Apply point loads to each floor
ops.load(5, Px, 0., 0.)    # Left 1. floor
ops.load(6, Px, 0., 0.)    # Middle 1. floor
ops.load(7, Px, 0., 0.)    # Middle 1. floor 2
ops.load(8, Px, 0., 0.)    # Right 1. floor

ops.load(9, Px, 0., 0.)    # Left 2. floor
ops.load(10, Px, 0., 0.)   # Middle 2. floor
ops.load(11, Px, 0., 0.)   # Middle 2. floor 2
ops.load(12, Px, 0., 0.)   # Right 2. floor

for etag in Ew:
    ops.eleLoad('-ele', etag, '-type', Ew[etag][0], Ew[etag][1],
```

```
                Ew[etag][2])

ops.constraints('Transformation')
ops.numberer('RCM')
ops.system('BandGeneral')
ops.test('NormDispIncr', 1.0e-6, 6, 2)
ops.algorithm('Linear')
ops.integrator('LoadControl', 1)
ops.analysis('Static')
ops.analyze(1)

ops.printModel()

opsv.plot_model()
plt.title('plot_model after defining elements')
plt.savefig('model_after_elements.png', dpi=300, bbox_inches='tight')
plt.clf()

opsv.plot_load()
plt.savefig('load_diagram.png', dpi=300, bbox_inches='tight')
plt.clf()

opsv.plot_reactions()
plt.savefig('reactions.png', dpi=300, bbox_inches='tight')
plt.clf()

# sfac = 80.

opsv.plot_defo()
plt.savefig('deformation.png', dpi=300, bbox_inches='tight')
plt.clf()
# opsv.plot_defo(sfac)
# fmt_interp = {'color': 'blue', 'linestyle': 'solid', 'linewidth': 1.2, 'marker':
    '.', 'markersize': 6}
# opsv.plot_defo(sfac, fmt_interp=fmt_interp)

# 4. plot N, V, M forces diagrams

sfacN, sfacV, sfacM = 3.e-5, 3.e-5, 3.e-5

opsv.section_force_diagram_2d('N', sfacN)
plt.title('Axial force distribution')
plt.savefig('axial_force.png', dpi=300, bbox_inches='tight')
plt.clf()

opsv.section_force_diagram_2d('T', sfacV)
plt.title('Shear force distribution')
plt.savefig('shear_force.png', dpi=300, bbox_inches='tight')
plt.clf()

opsv.section_force_diagram_2d('M', sfacM)
plt.title('Bending moment distribution')
plt.savefig('bending_moment.png', dpi=300, bbox_inches='tight')
plt.clf()

print("All figures are saved as PNG:")
print("- model_before_elements.png")
print("- model_after_elements.png")
print("- load_diagram.png")
print("- reactions.png")
print("- deformation.png")
print("- axial_force.png")
print("- shear_force.png")
print("- bending_moment.png")
```

```
exit()
```

Some of the outputs of the analysis can be seen in the following figures.

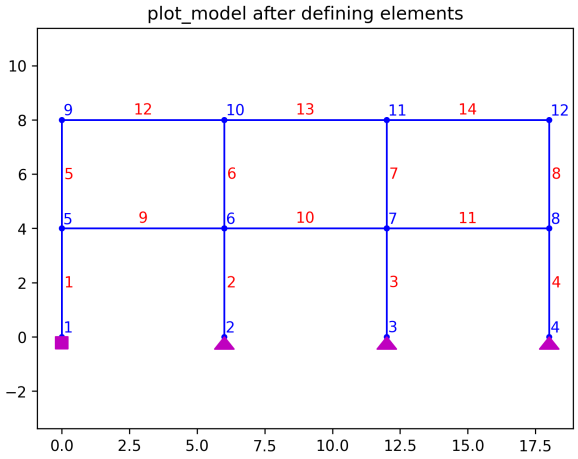


Figure 5: Geometry of the 2D frame.

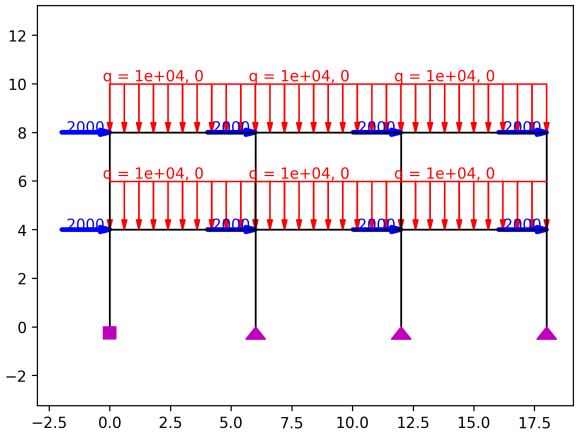


Figure 6: Load diagram of the 2D frame.

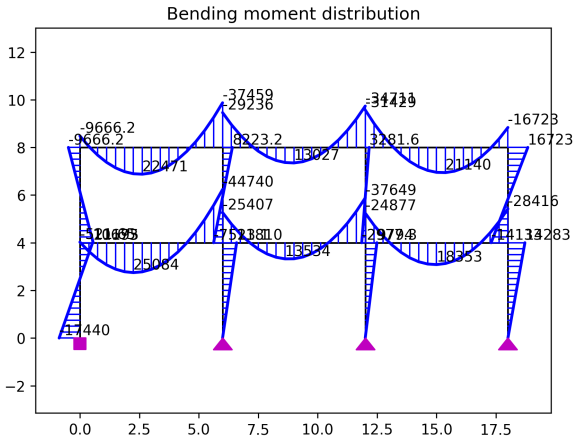


Figure 7: Bending moment diagram of the 2D frame.

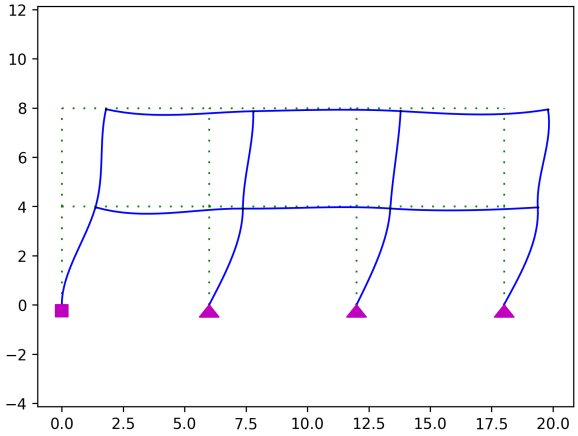


Figure 8: Deformation of the 2D frame.

16 Advanced Programming Concepts

This week introduces concepts that are foundational to understanding how modern software works. While we will not implement complex examples, knowing these ideas will help you read professional code and prepare you for any programming language (C++, Java, C#).

16.1 Memory and Pointers

In earlier weeks, we used variables as simple boxes that hold values. But how does a computer *actually* store data?

16.1.1 Memory as a Grid of Addresses

Computer memory (RAM) can be visualized as a long street of numbered post boxes. Each box has:

- A unique **Address** (e.g., 0x00A1, 0x00A2, ...)
- A **Value** (the actual data)

When you write `int x = 5;` in C# or C++, the computer:

1. Finds an empty box in memory (e.g., address 0x00A1).
2. Writes the value `5` into it.
3. Labels this box as `x` in its internal lookup table.

16.1.2 What is a Pointer?

A **Pointer** is a variable that stores an **Address**, not a value.

Analogy: Imagine you have a note (`ptr`) that says "Go to House #25". The note itself is not the house; it just tells you where the house is.

In C++:

```
int x = 10;          // A normal variable holding 10
int* ptr = &x;       // A pointer holding the ADDRESS of x
                     // & is the "address-of" operator
cout << *ptr;        // Output: 10. * is the "dereference" operator.
                     // It goes to the address and gets the value.
```

16.1.3 Why are Pointers Important?

- **Efficiency**: Instead of copying a gigabyte-sized array, you can just pass its address (a few bytes).
- **Low-Level Access**: Necessary for Operating Systems, Drivers, and Embedded Systems (e.g., a microcontroller in a sensor).
- **Data Structures**: Linked Lists, Trees, and Graphs require pointers to connect nodes.

16.1.4 Pointers in Python?

Python hides pointers from you. When you assign a list to another variable, you are copying a **reference** (an internal pointer), not the data itself.

```
list_a = [1, 2, 3]
list_b = list_a      # list_b now points to the SAME data
list_b[0] = 999
print(list_a)        # Output: [999, 2, 3]. Both changed!
```

This is why we sometimes need `copy.deepcopy()` to create a truly independent copy.

16.2 Object-Oriented Programming (OOP)

Until now, we wrote code as a sequence of instructions. This is **Procedural Programming**. For large projects (e.g., SAP2000, AutoCAD), this becomes unmaintainable. **OOP** organizes code around "Objects" that combine data and behavior.

16.2.1 Core Concept: Class and Object

- **Class**: A **Blueprint**. It defines what an object will look like.
- **Object**: An **Instance**. It is a real thing created from the blueprint.

Analogy: A "House Plan" (Class) vs. an "Actual House" (Object). You can build many houses from one plan.

```
// C# Example
public class Beam {
    public double Length; // Property (Data)
    public double Depth;

    // Method (Behavior)
    public double GetMomentOfInertia(double width) {
        return (width * Math.Pow(Depth, 3)) / 12;
    }
}

// Creating Objects (Instances)
Beam beam1 = new Beam();
beam1.Length = 5.0;
beam1.Depth = 0.5;
```

16.3 The Four Pillars of OOP

16.3.1 1. Encapsulation

Hiding the internal details of an object. You only expose what is necessary.

Analogy: A **Car**. You use the steering wheel and pedals (the public interface). You do not directly manipulate the fuel injectors (private internals).

In code, we use `private` and `public` keywords.

```
public class BankAccount {
    private double balance; // Cannot be accessed from outside

    public void Deposit(double amount) { // Public method
        if (amount > 0) balance += amount;
    }
}
```

16.3.2 2. Inheritance

Creating a **new class based on an existing one**. The new class inherits properties and methods from the parent.

Analogy: A **T-Beam** is a special type of **Beam**. It has everything a Beam has, plus a flange.

```
public class TBeam : Beam { // TBeam inherits from Beam
    public double FlangeWidth;

    // Override: We can change parent behavior
    public new double GetMomentOfInertia(double web) {
        // A more complex formula for T-sections
        return base.GetMomentOfInertia(web) + (FlangeWidth * 0.1);
    }
}
```

16.3.3 3. Polymorphism

"Many Forms". The **same command can behave differently** depending on the object type.

Analogy: The command "Draw yourself" given to different shapes. A **Circle** draws a circle. A **Rectangle** draws a rectangle. Same command, different results.

In code, this is achieved through method **overriding** (as shown above) or **interfaces**.

16.3.4 4. Abstraction

Focusing on **what an object does**, not **how it does it**.

Analogy: When you call `beam.CalculateDeflection()`, you don't care about the thousands of lines of Finite Element code running behind the scenes. You just want the answer.

In C#, this is often done with `abstract` classes or `interfaces`:

```
public abstract class StructuralElement {
    public abstract double GetWeight(); // No implementation here
}

public class Column : StructuralElement {
    public double Height;
    public override double GetWeight() {
        return Height * 2400 * 0.09; // Concrete density * area
    }
}
```

Note

Why OOP Matters for Engineers: Commercial engineering software (ETABS, ANSYS, Revit) are built with OOP. When you use APIs to automate these programs, you are interacting with their Objects (Nodes, Elements, LoadCases). Understanding OOP makes automation much easier.

16.4 Recursion

Recursion is when a function calls **itself**.

Analogy: Imagine a set of Russian Dolls (Matryoshka). To find the smallest doll, you open a doll, find another doll inside, and repeat until you reach the smallest one (the "base case").

```
def factorial(n):
    if n == 1:
        return 1            # Base Case: Stop here
    else:
        return n * factorial(n - 1)  # Recursive Call

print(factorial(5))  # 5 * 4 * 3 * 2 * 1 = 120
```

Engineering Use: Tree/Graph traversal algorithms, some optimization methods (Divide and Conquer).

Note

Warning: Every recursive function MUST have a **Base Case**. Otherwise, it will call itself forever and crash (Stack Overflow error).

16.5 Algorithm Complexity (Big O Notation)

Not all algorithms are equal. Some are fast, some are slow. **Big O Notation** describes how the runtime of an algorithm grows as the input size increases.

- **O(1) - Constant**: The operation takes the same time regardless of input size. (e.g., accessing an array element by index).
- **O(n) - Linear**: Time grows proportionally with input. (e.g., searching an unsorted list).
- **O(n^2) - Quadratic**: Time grows with the square of the input. (e.g., Bubble Sort). **Avoid for large data!**
- **O(log n) - Logarithmic**: Extremely efficient. (e.g., Binary Search).

Analogy: Finding a name in a phone book.

- O(n): Reading every page from the start ("linear search").
- O(log n): Opening the book in the middle, deciding if the name is before or after, then repeating ("binary search").

Why it matters: A $O(n^2)$ algorithm on 1,000 data points takes 1 million operations. On 1 million data points, it takes 1 **trillion** operations. Knowing this prevents you from writing code that never finishes.

16.6 Parallelism and Multithreading

Modern computers have multiple CPU cores. **Parallelism** means running multiple tasks at the same time on different cores.

Analogy: Building a house.

- **Sequential**: One worker does the foundation, then the walls, then the roof. Slow.
- **Parallel**: One team does foundation, another prepares wall materials simultaneously. Faster.

Engineering Use: Finite Element Analysis solvers (ANSYS, Abaqus) use parallelism to solve large stiffness matrices. Monte Carlo simulations run thousands of scenarios in parallel.

Note

Parallelism is powerful but tricky. If two "workers" try to modify the same data at the same time, you get a **Race Condition** bug, leading to unpredictable results.

16.7 Version Control (Git)

Version Control is a system that records changes to files over time. **Git** is the industry standard.

Analogy: An "Undo History" for your entire project, but you can also branch into alternate realities and merge them back.

Key Concepts:

- **Repository (Repo)**: The project folder tracked by Git.
- **Commit**: A snapshot of the project at a point in time. Like saving a game.
- **Branch**: A parallel version of the code. Used for developing new features without breaking the main code.
- **Merge**: Combining changes from one branch into another.
- **GitHub/GitLab**: Online platforms to host repositories and collaborate with others.

Why it matters: "My code was working yesterday, now it's broken" is solved by Git. You can always go back to a working version. It is also essential for team projects.

16.8 APIs (Application Programming Interfaces)

An **API** is a set of rules that allows one piece of software to talk to another.

Analogy: A **Waiter** in a restaurant. You (the customer/your code) don't go into the kitchen (the software's internal code). You tell the waiter (the API) what you want, and they bring you the result.

Engineering Examples:

- **SAP2000 OAPI**: Allows Python or C# to create models, run analysis, and read results programmatically.
- **Revit API**: Automates BIM modeling tasks.
- **Web APIs**: Get earthquake data from USGS, weather data for site planning, etc.

When you write `import openseespy`, you are using an API that talks to the underlying OpenSees engine.

16.9 Debugging and Testing

Debugging is the process of finding and fixing errors in code.

16.9.1 Common Debugging Techniques

- **Print Statements**: The oldest trick. Print variable values to see what's happening.
- **Debugger Tools**: IDEs (VS Code, PyCharm) let you pause execution ("breakpoints"), step through code line by line, and inspect variables.
- **Rubber Duck Debugging**: Explain your code line by line to an inanimate object (or a colleague). Often, you find the bug while explaining.

16.9.2 Testing

Professional software includes **Unit Tests**: small functions that check if individual pieces of code work correctly.

```
# A simple unit test
def test_factorial():
    assert factorial(5) == 120
    assert factorial(1) == 1
    print("All tests passed!")
```

Why it matters: Code without tests is like a building without inspection. You *think* it's correct, but you don't *know*.

Notes